

1. Operating System (OS) kya hai aur iska Position

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference Note: Yeh topic aapke dwara provide ki gayi **Image 1** par aadharit hai jisme OS ki definition, concentric position diagram aur computer system ke 4 components explain kiye gaye hain.

1.1 Operating System ki Basic Definition

- **Master Program:** Operating System ek aisa **program / system software** hai jo computer ke physical hardware (H/W) ko poori tarah manage aur control karta hai.
- **Intermediary / Bridge Role:** Yeh user aur raw computer hardware ke beech ek **intermediary (madhyasth)** ka kaam karta hai. Agar OS na ho, toh user ko computer ke binary circuits, registers aur memory buses se direct baat karni padegi jo aam insaan ke bas ki baat nahi hai.
- **Basis for Application Programs:** OS ek aisa basic foundation provide karta hai jiske upar saare application software (jaise Web Browser, MS Office, Code Editors, Games) install hokar run kar sakte hain.
- **User-Friendly Environment:** OS ek convenient aur easy-to-use environment taiyar karta hai jisme user bina kisi technical complexity ke apne programs execute kar sake.

1.2 Diagram Reference & Handwritten Notes (Image 1)

Original Image 1 mein ek concentric circle (onion-like) diagram diya gaya hai jiska title hai Fig: - Position of Operating System. Image 1 ke right side mein 3 items/entities note kiye gaye hain:

Fig 1.1: Position of Operating System (Concentric View)

Reference: Image 1 [Position of OS & 3 Entities Architecture]

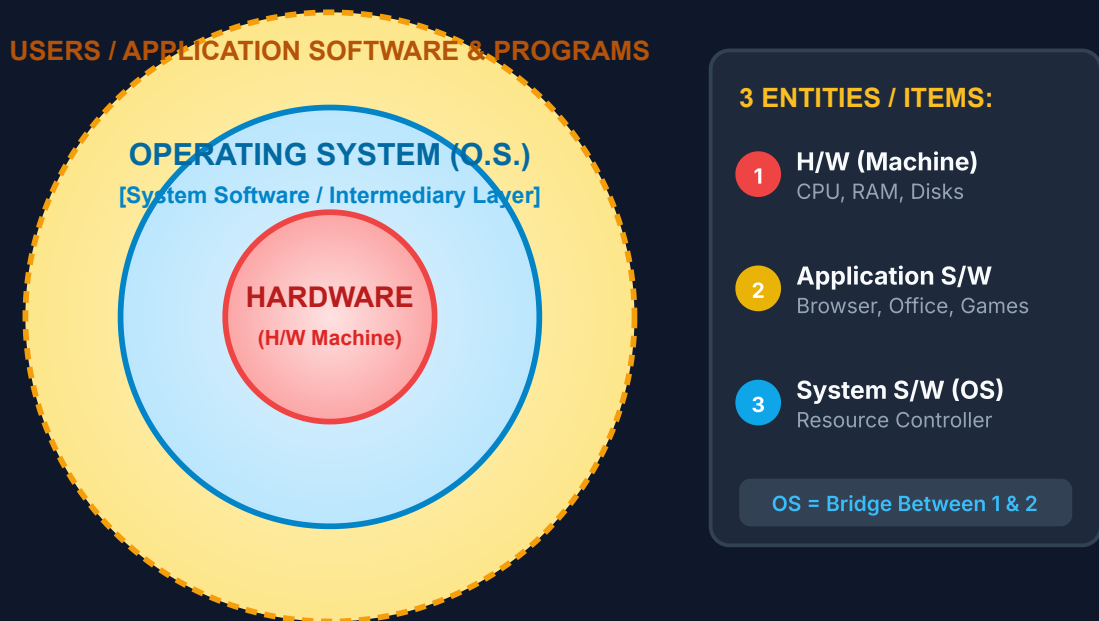


Fig 1.1: Position of Operating System & 3 Core Entities Architecture

1.3 Computer System ke 3 Main Entities (Items)

- **1.3.1 Hardware (H/W - Machine):** Physical components (CPU, RAM, Hard Disk, Motherboard, Graphics Card). Yeh computing capability aur raw processing power provide karta hai.
- **1.3.2 Application Software (Application S/W):** Specific end-user problems ko solve karne ke liye banaye gaye programs (e.g., Browser, Word Processor, Media Player, Games).
- **1.3.3 System Software (OS & Utilities):** Hardware aur application software ke beech coordination banata hai aur resource allocation control karta hai (~50% core management share).

1.4 Computer System ke 4 Core Components

Computer system ko 4 mukhya bhaagon mein divide kiya jata hai:

1. **The Hardware (H/W):** Basic computing resources provide karta hai (CPU, Memory, I/O Devices).
2. **The Operating System (O.S.):** Hardware ke use ko control aur coordinate karta hai different users aur application programs ke beech.
3. **The Application Programs:** Define karta hai ki hardware resources ka use karke user ki computing problems ko kaise solve kiya jaye.
4. **The Users:** End-users jo computer ko use karte hain (Humans, remote machines, ya automated background bots).

2. Computer System ka Abstract View aur Component Hierarchy

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference to Previous Concept: Jaise humne **Concept 1.4** mein dekha ki computer system ke 4 components hote hain (Hardware, OS, Application Programs, Users), yeh topic **Image 2** par aadharit hai jo in components ka layered **Abstract View** aur right side mein **Translators** ka flow explain karta hai.

2.1 Abstract View of Components (Layered Architecture)

- **Layer 1 (Bottom Most) - Computer Hardware (H/W):** Core machine jisme CPU, Main Memory (RAM), aur I/O devices aate hain.
- **Layer 2 - Operating System (OS):** Hardware ke theek upar rehta hai aur hardware ko wrap karke safe, uniform services deta hai.
- **Layer 3 - System & Application Programs:** OS ke upar different utilities run hoti hain: COMPILER, ASSEMBLER, TEXT EDITOR, DATABASES, SPREADSHEETS, GAMES.
- **Layer 4 (Top Most) - Multiple Users:** User 1, User 2, ... User N jo concurrently system use kar rahe hote hain.

2.2 Translators (Handwritten Note Reference from Image 2)

Image 2 ke top-right corner mein likha hai: TRANSLATORS :- 1) ASSEMBLER 2) COMPILER, INTERPRETER | HLL -> 0, 1.

- **Translators ki Zaroorat:** Computer Hardware (CPU) sirf aur sirf **Machine Language (0 aur 1 / Binary Bits)** samajhta hai. Hum insaan high-level English-like code likhte hain (C, C++, Java, Python). Translators ka kaam iss Human-readable code ko Computer-readable binary bits (0, 1) mein convert karna hai.
- **2.2.1 Assembler:** Assembly Language (Low-Level Language jaise MOV AX, BX) ko seedhe Machine Language (0 aur 1) mein convert karta hai.
- **2.2.2 Compiler:** High-Level Language (HLL jaise C/C++) ke poore code ko ek saath compile karke machine code generate karta hai.
- **2.2.3 Interpreter:** High-Level Language (jaise Python, JS) ko line-by-line interpret karke execute karta hai.

Fig 2.1: Abstract View of Computer System Components

Reference: Image 2 [Layered Components & Translators Architecture]

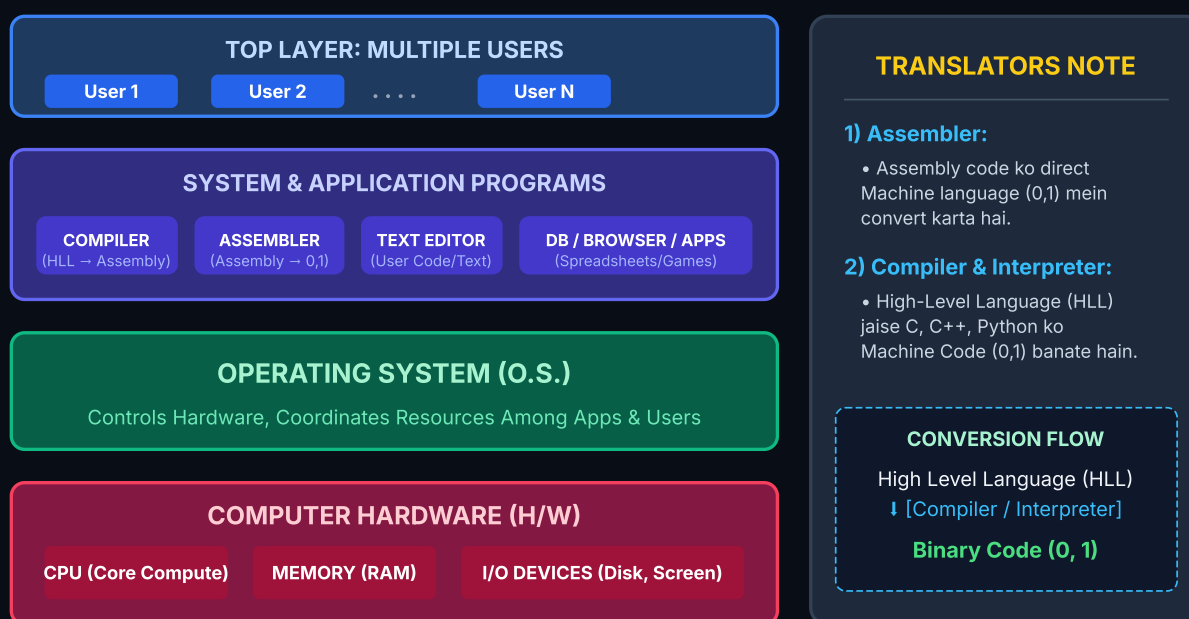


Fig 2.1: Abstract View of Computer System & Translators Flow

2.3 Hardware Resources (H/W Deep Dive)

- **(a) The Memory (RAM):** Programs ko execute hone ke liye memory chahiye hoti hai. Variables aur active data yahin store hota hai.
- **(b) Input & Output (I/O) Devices:** Disks (Storage), Keyboard, Mouse, Monitor, Network cards jinke zariye data in aur out hota hai.
- **(c) CPU (Central Processing Unit):** Basic computing resources aur calculations execute karne wala brain.

2.4 Application Programs vs OS Coordination

Application programs (Word, Spreadsheets, Games, DBMS) solve user computing problems. OS in programs ke beech hardware resources ka coordination karta hai taaki koi application hardware ko monopolize na kare aur system stable bana rahe.

3. User's aur System's Point of View of Operating System

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference to Previous Concepts: Concept 1.2 mein OS ko intermediary aur Concept 2.1 mein Abstract View dekha gaya tha. Yeh topic **Image 3** par aadharit hai jisme OS ko do mukhya perspectives se dekha jata hai: **User View** aur **System View (Resource Manager)**.

3.1 User View (End-User Perspective)

- **Primary Goal: Convenience, Usability & Responsiveness:** User ke liye OS ka matlab hai aasan aur bina rukawat computer chalana.
- **Interface Modes:** GUI (Windows, icons), CLI (Terminal commands), aur APIs (Developers ke liye).
- **User Focus:** Running applications, managing files, configuring settings, and interacting with peripherals.

3.2 System View (Resource Manager Perspective)

Image 3 handwritten note: "**Resource Manager**" (marked very important).

- **Primary Goal: Efficiency, Control & Resource Allocation:** System perspective se OS ek Resource Allocator hai. Hardware limited hai (CPU cores, RAM, I/O bandwidth). OS ka kaam hai fair aur collision-free distribution karna.
- **Core Components:** Kernel, Device Drivers, Memory Management, Process Management, File Systems, Networking Protocols.
- **Emphasis:** Efficiency, fair resource allocation, security aur stability.

Fig 3.1: Dual Perspectives of Operating System

Reference: Image 3 [User View vs System View / Resource Manager]

1. USER VIEW (End-User)

Primary Goal: Convenience & Usability

- Platform for executing applications
- Interfaces: GUI, CLI, APIs
- Focus: Running apps, files, settings
- User concerns: Responsiveness, ease of use

Example: User Perspective

User writes code:

```
a = b + c;
```

User only cares about output correctness and fast execution!

2. SYSTEM VIEW (Resource Manager)

Primary Goal: Efficiency & Control

- OS as a master Resource Allocator
- Controls CPU, Memory, I/O devices
- Enforces Security, Stability & Isolation
- Target audience: SysAdmins, Engineers

Example: System Perspective

Inside OS & Hardware:

1. Memory allocated for a, b, c
2. Values loaded in registers (R1, R2)
3. CPU executes ADD, result written to RAM
4. Protection prevents illegal memory access

Fig 3.1: Dual Perspectives of Operating System

3.3 Numerical / Computational Example & Step-by-Step Thought Process: $a = b + c$

Image 3 handwritten note: $a = b + c$. Aaiye iska complete thought process samjhein:

Scenario: User writes `int b = 10; int c = 20; int a = b + c;`

- User View Thought Process:** User chahta hai ki `10 + 20` add hokar turant `30` mil jaye. User ko underlying hardware address se matlab nahi hai.
- Translators Role (Ref Concept 2.2):** Compiler ne iss instruction ko machine code mein toda:
 - `LOAD R1, [Addr_B]` (Value 10 in Register R1)
 - `LOAD R2, [Addr_C]` (Value 20 in Register R2)
 - `ADD R3, R1, R2` (ALU performs addition: 30)
 - `STORE [Addr_A], R3` (Value 30 stored in RAM address A)
- System View (Resource Manager) Thought Process:**
 - Process Allocation:* OS iss code ko PID deta hai aur isolated RAM allot karta hai.
 - Memory Protection:* MMU check karta hai ki process apni boundary ke bahar na jaye.
 - CPU Scheduling:* OS process ko CPU cycle allot karta hai aur context switch ke samay register state save karta hai.
 - I/O Signaling:* Final output screen driver ke zariye user ko deliver hota hai.

3.4 User View vs System View Comparison Table

Feature	User View (End-User)	System View (Resource Manager)
Main Objective	Convenience & Usability	Efficiency & Resource Optimization
Interaction Mode	GUI, CLI, Desktop Apps	System Calls, Interrupts, Drivers
Focus Area	App execution & output response	Memory Allocation, CPU Scheduling, Security
Perspective on $a=b+c$	"Sirf 30 answer aana chahiye."	"RAM allocate hui, Registers load hue, MMU verified!"

4. Kernel aur uske Core Operations (With Types of Kernel)

Computer Science Fundamentals | Module 1: OS Architecture | CS_Fundamental-00

Reference to Previous Concepts: Concept 1.1 mein humne dekha tha ki OS hardware manage karta hai aur Concept 3.2 (System View) mein Kernel ko core component bataya gaya tha. Yeh topic **Image 4** par aadharit hai jo Kernel ke **5 Fundamental Operations** aur **3 Types of Kernel** ko detail karta hai.

4.1 Kernel kya hota hai (The Core Bridge)

- **Heart of the OS:** Kernel OS ka core program hai jo boot hote hi RAM mein load hota hai aur system chalne tak active rehta hai.
- **Bridge Role:** Yeh hardware aur software ke beech direct bridge hai. Applications hardware ko direct touch nahi kar sakte; unhe Kernel se **System Calls** ke through permission leni padti hai.

4.2 Key Functions / Operations of Kernel (5 Pillars)

1. **Process Management:** CPU time allot karna (CPU Scheduling), process creation, coordination aur termination ensure karna.
2. **Memory Management:** RAM allocation/deallocation, Memory Protection (isolation), aur Virtual Memory handle karna.
3. **Device Management:** Hardware peripherals ke device drivers handle karna aur uniform I/O interface provide karna.
4. **File System Management:** Storage devices (SSD/HDD) par files create, read, write aur organize karna.
5. **Security & Access Control:** User permissions verify karna, unauthorized access rokna aur system integrity maintain karna.

Fig 4.1: Kernel Architecture & Core Operations

Reference: Image 4 [Kernel Functions & 3 Types Comparison]

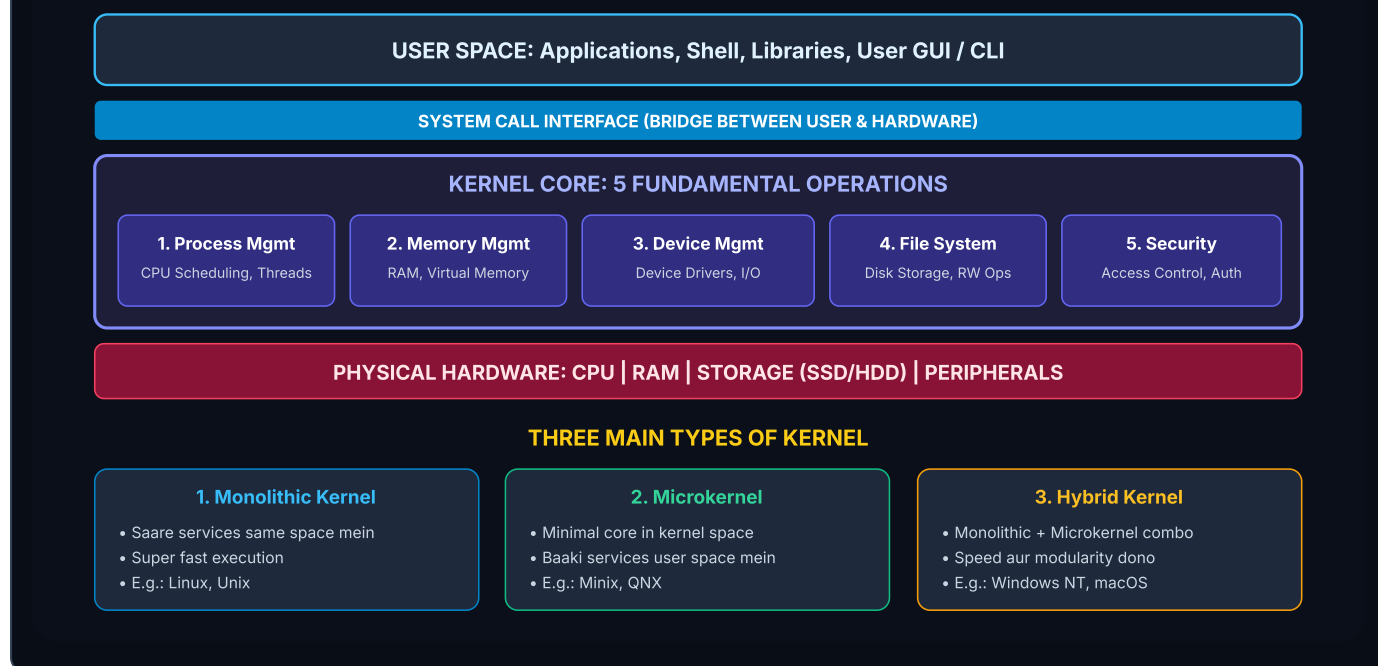


Fig 4.1: Kernel Architecture, 5 Operations & 3 Kernel Types

4.3 Types of Kernel (Image 4 Reference)

4.3.1 Monolithic Kernel

- Definition:** Saare OS services (Process, Memory, File System, Drivers) **ek hi address space (Kernel Space)** mein run karte hain.
- Pros:** Super fast execution (direct memory function calls).
- Cons:** Agar ek driver crash ho, toh poora OS crash ho sakta hai.
- Examples:** Linux, Traditional Unix.

4.3.2 Microkernel

- Definition:** Kernel space ko **bahut chhota (minimal)** rakhta hai. Baaki services User Space mein servers ke roop mein chalti hain.
- Pros:** Highly secure aur crash-resistant (Driver crash hone se OS nahi marta).
- Cons:** IPC (message passing) overhead ki wajah se thoda slow.
- Examples:** Minix, QNX, L4.

4.3.3 Hybrid Kernel

- Definition:** Monolithic ki high speed aur Microkernel ki modularity ko combine karta hai.
- Examples:** Windows NT (Windows 10/11), macOS (XNU).

4.4 Kernel Types Comparison Table

Paimana	Monolithic Kernel	Microkernel	Hybrid Kernel
---------	-------------------	-------------	---------------

Address Space	Saare services Kernel Space mein	Sirf core kernel space mein, baaki User Space	Mixture (Critical parts in kernel space)
Speed	Bahut Fast	Thoda Slow (IPC overhead)	Fast & Balanced
Fault Tolerance	Low (Driver crash = OS crash)	Very High (Stable)	High
Real Examples	Linux, Unix	Minix, QNX	Windows NT, macOS

5. Computer System Organization & Architecture

Computer Science Fundamentals | Module 2: System Organization | CS_Fundamental-00

Reference to Previous Concepts: Concept 2.1 & 2.3 mein humne Hardware resources (CPU, Memory, I/O) dekhe the. Yeh topic **Image 5** par aadharit hai jisme modern system ka **Common Bus Architecture, Device Controllers**, aur bottom slide mein **CPU Internal Architecture (ALU, CU, Registers, I/O Modules)** explain kiye gaye hain.

5.1 Modern Computer System Operation & Common Bus

- **Structure:** Modern general-purpose computer system consists of one or more CPUs and a number of device controllers connected through a **Common System Bus** that provides access to shared memory.
- **Multicore Architecture (Image 5 Note):** Single chip par multiple execution cores hone se parallel computing aur high throughput milta hai.

5.2 Device Controllers & Concurrent Execution

- **Specialized Controllers:** Disk Controller (Storage), USB Controller (Peripherals), Graphics Adaptor (Video output).
- **Memory Cycle Competition:** CPU aur Device Controllers **concurrently (ek sath)** execute hote hain aur shared memory access ke liye compete karte hain.

5.3 Memory Controller & Synchronization

Orderly access ensure karne ke liye **Memory Controller** provide kiya jata hai jo memory access ko synchronize karta hai aur data clash rokta hai.

Fig 5.1: Computer System Organization & CPU Architecture

Reference: Image 5 [Modern System Bus & Internal CPU Operations]

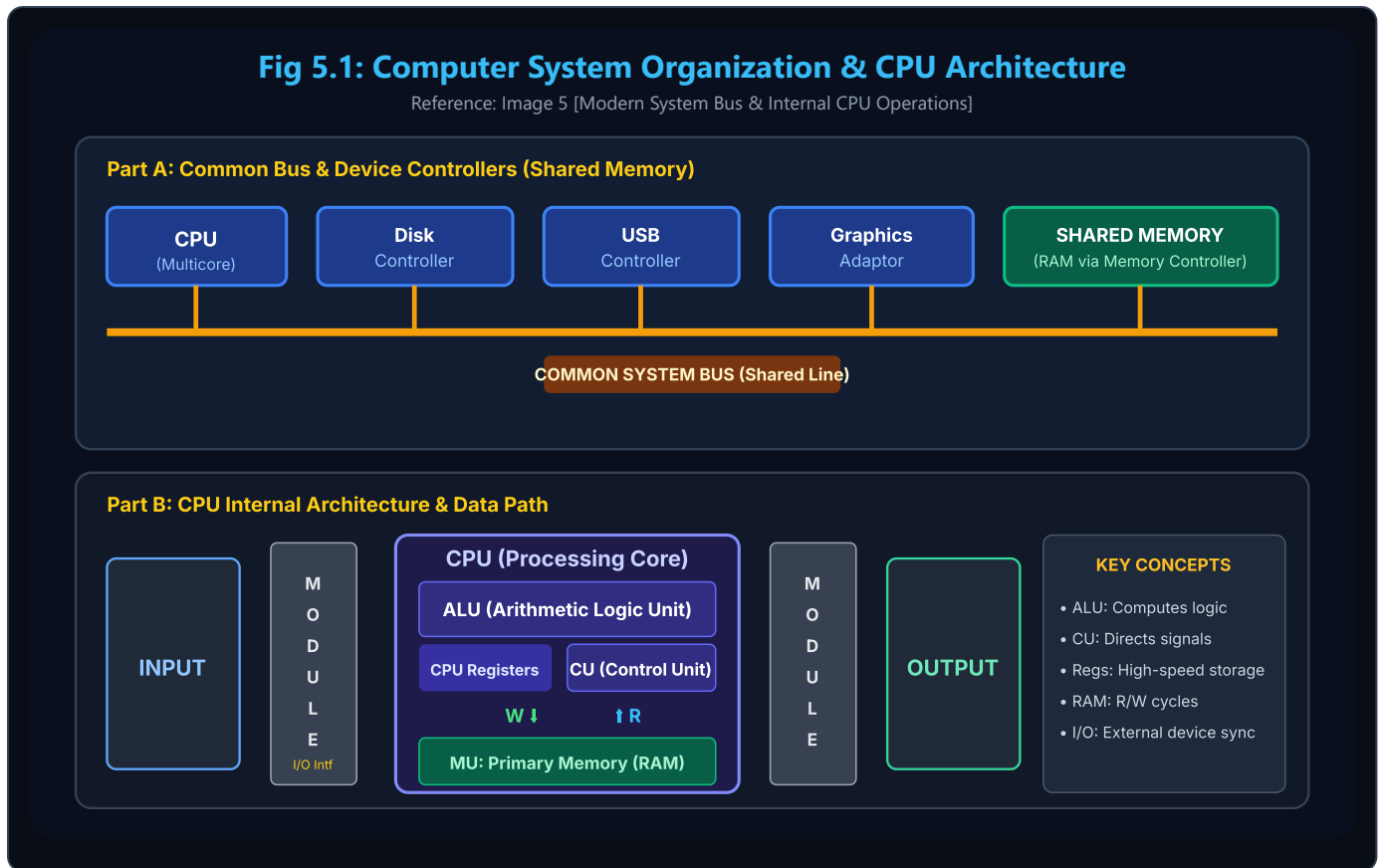


Fig 5.1: Common Bus Architecture & CPU Internal Data Path

5.4 CPU Internal Architecture (Image 5 Bottom Handwritten Diagram)

- **ALU (Arithmetic Logic Unit):** Arithmetic operations (+, -, *, /) aur logical decisions (AND, OR, NOT).
- **CU (Control Unit):** Data timing aur instruction sequencing control karta hai.
- **CPU Registers:** High-speed internal storage for immediate data.
- **Memory Unit (MU / RAM):** Read (R) aur Write (W) operations.
- **I/O Interface Modules:** Input data receive karna aur processed output display/devices tak bhejna.

6. Bootstrap Program aur Booting Process

Computer Science Fundamentals | Module 2: System Booting | CS_Fundamental-00

Reference to Previous Concepts: Concept 4.1 mein humne dekha tha ki Kernel OS ka core engine hai jo boot par RAM mein load hota hai. Yeh topic **Image 6 & Image 7** par aadharit hai jo batata hai ki RAM khaali hone par Kernel ko kaun aur kaise load karta hai!

6.1 Bootstrap Program (Bootstrap Loader) kya hai?

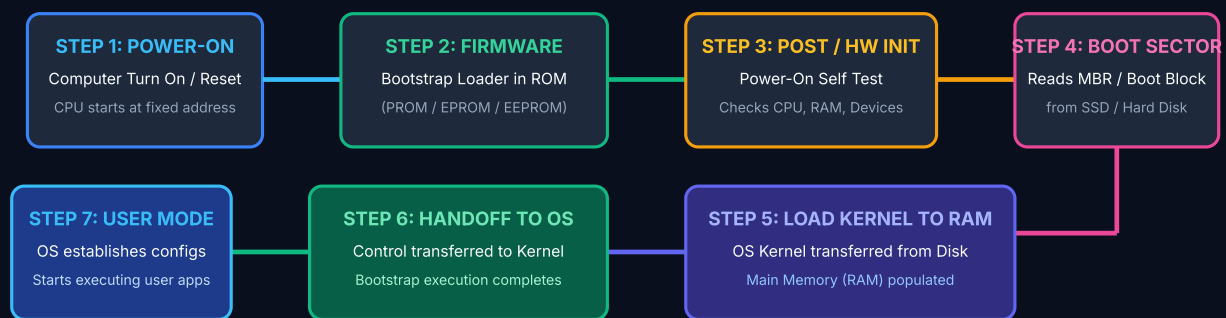
- **Initial Startup Code:** Chhota sa code jo non-volatile **Firmware / ROM** mein store hota hai.
- **Primary Purpose:** System hardware initialize karna aur storage drive se OS Kernel ko Main Memory (RAM) mein load karke CPU ka control usse sonpna.

6.2 Firmware & ROM Types (Image 6 Handwritten Note)

- **ROM:** Permanent Read-Only Memory.
- **PROM / EPROM:** Programmable aur UV-erasable ROM chips.
- **EEPROM / Flash:** Electrically Erasable Programmable ROM (Modern BIOS / UEFI chips).

Fig 6.1: Bootstrap Process & System Booting Sequence

Reference: Image 6 & 7 [Firmware, POST, Boot Block, Kernel Handoff]



Summary of Firmware Terminology (Image 6 Note):

ROM (Read Only Memory): Permanent non-volatile storage jisme initial bootstrap code rehta hai. **PROM / EPROM / EEPROM:** Programmable / Erasable ROM chips jisme firmware updates flash kiye jate hain (BIOS / UEFI). **Boot Sector / Boot Block:** Storage device ka first physical sector (MBR/GPT) jo OS bootloader code rakhta hai. **Kernel Handoff:** Ek baar Kernel RAM mein load hua, firmware CPU ka control Kernel ko de deta hai aur kaam khatam kar deta hai.

6.3 Step-by-Step Working of Bootstrap Program (Images 6 & 7)

1. **Power-On or Reset:** CPU fixed startup routine se instruction fetch karna shuru karta hai.
2. **POST (Hardware Diagnostics):** CPU registers, memory aur peripheral devices ki integrity check hoti hai.
3. **Loading the OS:** Boot device (SSD/HDD) ke **Boot Sector / Boot Block** ko read karke Kernel ko RAM mein transfer karta hai.
4. **Handoff to Kernel:** Kernel RAM mein aate hi CPU execution control Kernel ko handoff kar diya jata hai.
5. **User Mode Transition:** OS system configurations establish karta hai aur user applications chalane ke liye **User Mode** mein enter kar jata hai.

7. Dual-Mode Operation (User Mode vs Kernel Mode)

Computer Science Fundamentals | Module 2: OS Protection | CS_Fundamental-00

Reference to Previous Concepts: Concept 3.1 & 3.2 mein User View vs System View dekha tha, aur Concept 4.1 mein Kernel ko bridge dekha tha. Yeh topic **Image 7 (bottom) & Image 8 (top)** par aadharit hai jo **User Mode vs Kernel Mode** aur code `printf("Hello");` ka transition explain karta hai.

7.1 Dual-Mode Operation ki Zaroorat (Protection & Safety)

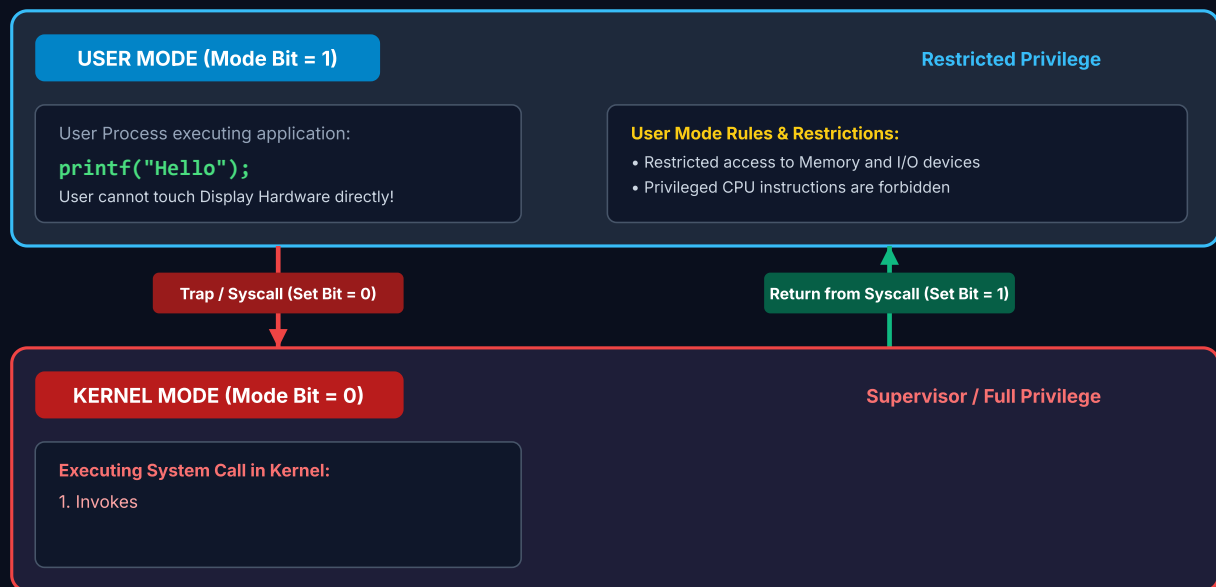
Agar koi normal software hardware ya RAM ko directly modify kar sake, toh ek glitch poore computer ko crash kar sakta hai. Isliye hardware do distinct privilege modes provide karta hai.

7.2 User Mode vs Kernel Mode

- **User Mode (Restricted Privilege - Mode Bit = 1):** User processes execute hote hain. Hardware resources direct access nahi kiye ja sakte.
- **Kernel Mode (Supervisor / Full Privilege - Mode Bit = 0):** OS Kernel execute hota hai. Hardware, RAM, registers aur privileged instructions ka full access hota hai.

Fig 7.1: Dual-Mode Operation & System Call Transition

Reference: Image 7 (Bottom) & Image 8 (Top) [User Mode → Kernel Mode via printf]



`sys_write()` 2. Sends data to Video Buffer / Display 3. Writes "Hello" to Terminal Screen Kernel Mode Privileges: • Full control over hardware resources & memory • Direct execution of privileged machine instructions • Manages hardware interrupts, I/O ports & MMU

Fig 7.1: Dual Mode Transition & System Call Trace

7.3 Handwritten Example & Step-by-Step Thought Process: printf("Hello");

Image 7 bottom note: USER MODE -----> Kernel mode | printf("Hello");

Thought Process Trace:

1. User Mode Execution (Mode Bit = 1):

User program `printf("Hello")` call karta hai. Library pehchanti hai ki display screen par write karna ek hardware operation hai jo User Mode mein allowed nahi hai.

2. Trap & Mode Switch (1 → 0):

Program `sys_write()` system call execute karta hai. Hardware Trap trigger hota hai aur CPU Mode Bit ko `1` se `0` (Kernel Mode) switch kar deta hai.

3. Kernel Mode Execution (Mode Bit = 0):

Kernel parameters verify karta hai aur display controller ke video buffer mein safely write karta hai. Screen par "Hello" print ho jata hai.

4. Return to User Mode (0 → 1):

Kernel return instruction deta hai. Hardware Mode Bit ko wapas `1` (User Mode) set kar deta hai aur user app aage chalta hai.

7.4 User Mode vs Kernel Mode Comparison Table

Paimana	User Mode	Kernel Mode
Mode Bit	1	0
Privilege	Restricted Access	Full Hardware & Memory Access
Executing Code	User Applications (Chrome, Word, Python)	Operating System Kernel
Hardware I/O	Direct I/O Forbidden (Needs System Call)	Direct I/O Permitted

8. Functions of Operating System: Memory Management

Computer Science Fundamentals | Module 2: Memory Management | CS_Fundamental-00

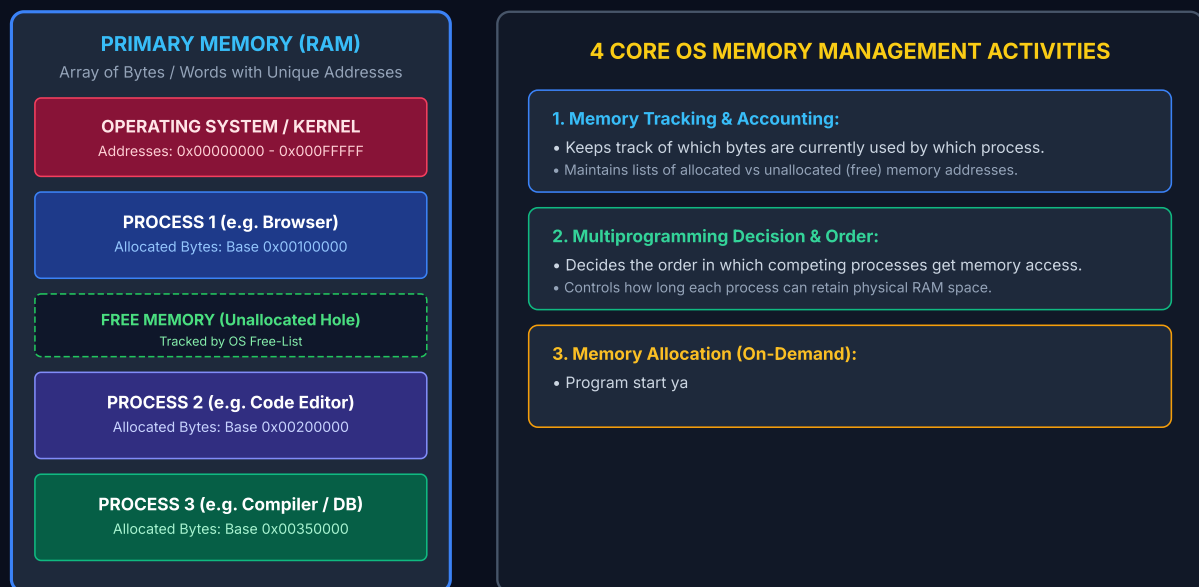
Reference to Previous Concepts: Concept 4.2.2 mein humne Kernel ke 5 core functions mein Memory Management dekha tha, aur Concept 3.3 ke numerical example ($a = b + c$) mein RAM allocation samjha tha. Yeh topic **Image 8 (bottom slide)** par aadharit hai.

8.1 Primary Memory / Main Memory (RAM) ki Sanrachna

- **Array of Bytes / Words:** Main memory ek bahut bada array of bytes/words hoti hai jisme har byte/word ka ek unique numerical memory address hota hai.
- **Direct CPU Access:** Fast storage jise CPU system bus ke zariye direct read/write access karta hai.
- **Execution Shart:** Kisi bhi program ko execute hone ke liye uska pehle **Main Memory (RAM)** mein load hona anivarya (mandatory) hai.

Fig 8.1: Main Memory Array Architecture & OS Management Activities

Reference: Image 8 [Primary Memory Array of Bytes, Multiprogramming & Allocation]



`malloc()` call par RAM bytes allot karta hai. • Sets Base and Limit registers in MMU for safety. 4. **Memory Deallocation & Reclamation:** • Process exit hone par ya I/O wait ke dauran memory free karta hai. • Reclaims blocks into free-list to prevent Memory Leaks.

Fig 8.1: Primary Memory Array & 4 Core OS Activities

8.2 OS ki 4 Core Memory Management Activities (Image 8 Reference)

1. **Memory Tracking & Accounting:** Track karta hai ki kaun se bytes kis process dwara use ho rahe hain, kaun se allocated hain aur kaun se free hain.

2. **Multiprogramming Access Decisions:** Multiprogramming environment mein decide karta hai ki competing processes ko kis order mein aur kitni der tak memory access milega.
3. **Dynamic Memory Allocation:** Process request par RAM bytes allot karta hai aur MMU base/limit registers set karta hai.
4. **Memory Deallocation & Reclamation:** Process terminate hone par memory reclaim karta hai taaki Memory Leaks na hon.

8.3 Numerical / Computational Example & Step-by-Step Thought Process: Memory Addressing & Allocation

Scenario: Total RAM = 1024 KB. OS Reserved = 0-199 KB (200 KB). Free = 200-1023 KB (824 KB).

1. **Process P1 Request (300 KB):** OS P1 ko 200 se 499 KB allot karta hai. Base Register = 200 KB, Limit Register = 300 KB. Free memory bachi = 524 KB (500-1023 KB).
2. **Process P2 Request (250 KB):** OS P2 ko 500 se 749 KB allot karta hai. Base Register = 500 KB, Limit Register = 250 KB. Free memory bachi = 274 KB.
3. **Hardware Protection Verification:** Agar P2 address 450 KB access karne ki koshish kare, toh MMU hardware check karega: $450 < \text{Base}(500)$ → **Illegal Access!** Trap trigger karke process kill ho jayega (Memory Protection!).
4. **Deallocation:** P1 terminate hone par 300 KB block free-list mein return hota hai aur agle processes ke liye ready ho jata hai.

9. Memory Hierarchy & Caching Deep Dive

Computer Science Fundamentals | Module 3: Memory Subsystem | CS_Fundamental-00

Interview Memory Hook: "Jo memory jitni fast hogi, woh utni hi mehngi aur chhoti hogi. Jo memory sasti aur badi hogi, woh dheemi hogi." Is trade-off ko solve karne ke liye **Memory Hierarchy** aur **Multi-Level Cache** bani!

9.1 Memory Hierarchy Pyramid & Core Trade-offs

- **Level 1 (Top) - CPU Registers:** Fastest (0.25-0.5 ns), small (few KB), expensive.
- **Level 2 - Cache Memory (SRAM):** L1, L2, L3 caches (1-20 ns).
- **Level 3 - Main Memory (DRAM):** Primary RAM (50-100 ns, 8-64 GB), volatile.
- **Level 4 - Secondary Storage:** NVMe / SSD / HDD (Microseconds to milliseconds, Terabytes).
- **Level 5 (Bottom) - Magnetic Disks & Cold Storage:** Backup / Tapes.

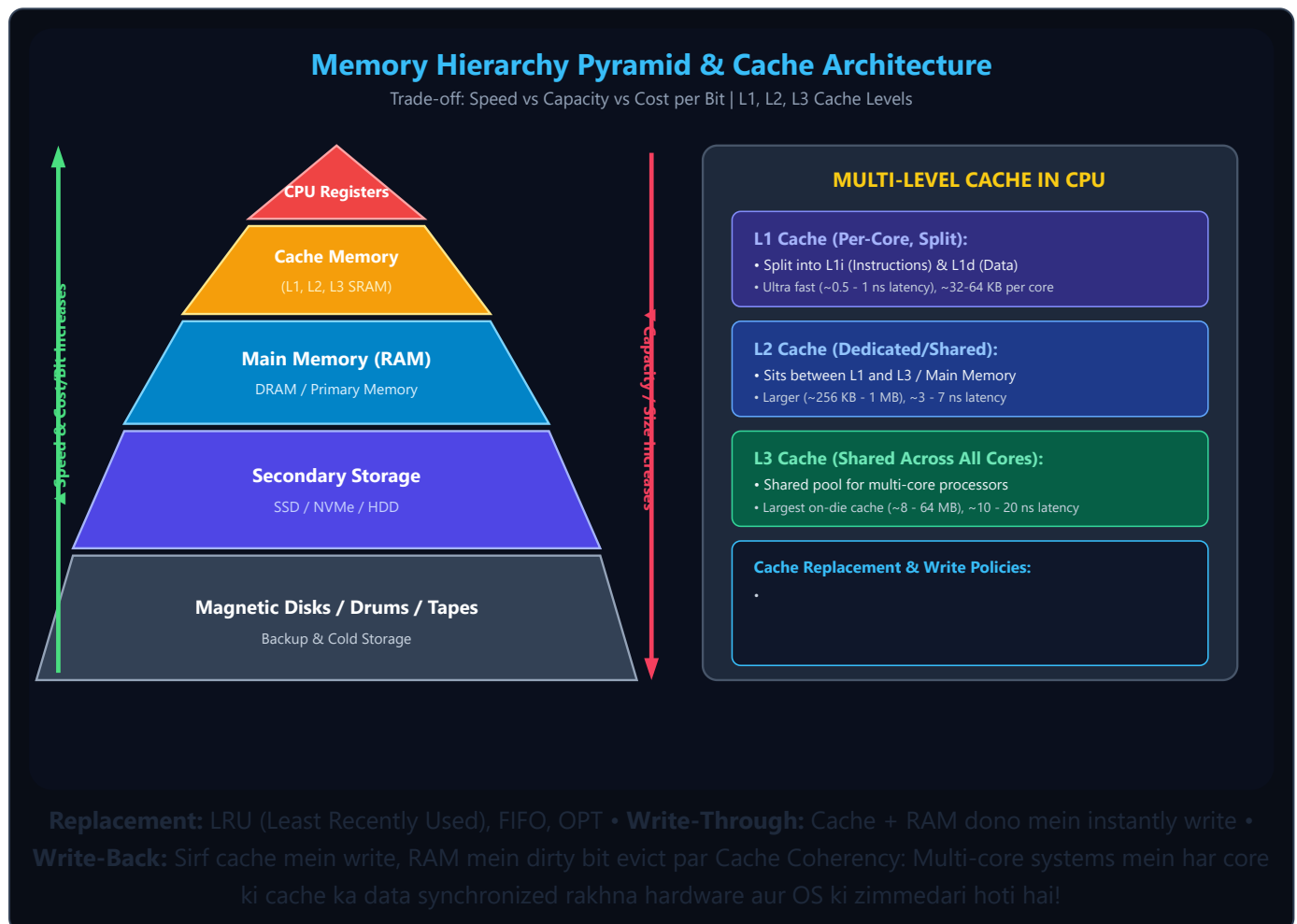


Fig 9.1: Memory Hierarchy Pyramid & Multi-Level Cache Architecture

9.2 Caching & Principle of Locality

- **Temporal Locality:** Jo data abhi access hua hai, woh jald hi dobara access hoga (Loops, counters).

- **Spatial Locality:** Jo memory address access hua hai, uske agal-bagal ke addresses bhi jald access honge (Arrays, sequential code).
- **Average Memory Access Time (AMAT):** $AMAT = T_{cache} + (1 - H) * T_{RAM}$

9.3 Multi-Level Caches (L1, L2, L3)

- **L1 Cache:** Per-core dedicated, split into L1i (Instructions) & L1d (Data). ~32-64 KB, ~1 ns.
- **L2 Cache:** Intermediate buffer between L1 and RAM. ~512 KB - 2 MB per core.
- **L3 Cache:** Shared across all CPU cores on die. ~8 - 64 MB, ~10-20 ns.

9.4 Cache Policies: Replacement & Writes

- **Replacement:** LRU (Least Recently Used), FIFO, Random, OPT.
- **Write-Through:** Cache aur RAM dono mein instantly write (consistent, slower).
- **Write-Back:** Sirf cache mein write, Dirty Bit set. RAM mein evict hone par likhta hai (fastest).
- **Cache Coherency:** Multi-core mein sabhi cores ke cache data ko synchronize rakhna (MESI protocol).

10. Processor Management & CPU Scheduling

Computer Science Fundamentals | Module 3: Process Subsystem | CS_Fundamental-00

Interview Memory Hook: Program hard disk par rakhi ek band kitaab hai. Jab use khol kar execute karte hain, tab woh **Process** banti hai! OS ka Processor Manager decide karta hai ki kaun sa process kab tak CPU core par chalega.

10.1 Program vs Process & PCB

- **Program:** Passive code file on disk (app.exe).
- **Process:** Active entity in execution with Program Counter, Stack, Heap, Registers, and PCB.
- **PCB (Process Control Block):** Process ID, State, CPU Registers, Memory limits.

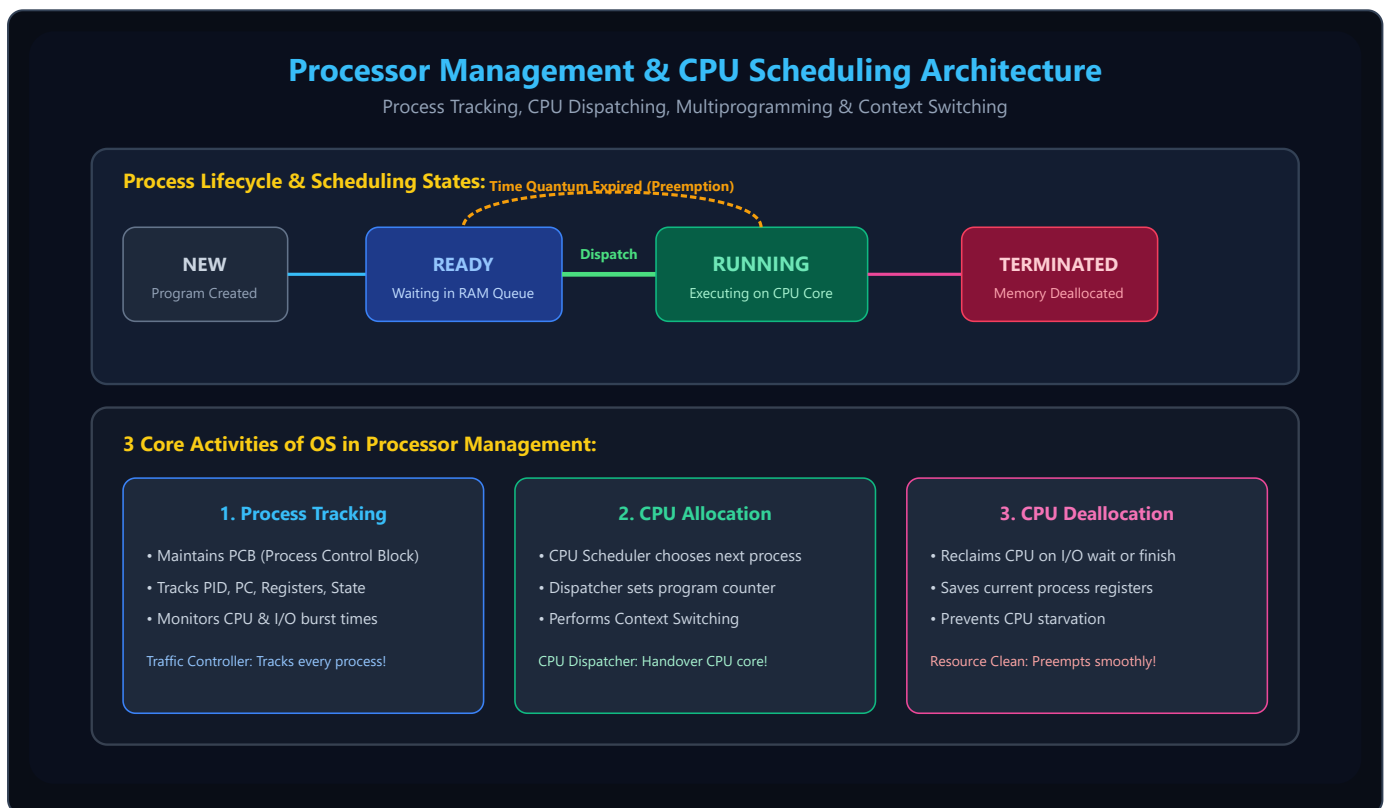


Fig 10.1: Process Lifecycle & CPU Scheduling Dispatcher

10.2 Multiprogramming & Scheduling

CPU utilization maximize karne ke liye jab chal raha process I/O wait karta hai, OS turant doosre ready process ko CPU dispatch kar deta hai.

10.3 OS ki 3 Core Activities

1. **Process Tracking:** PCB aur process states (New, Ready, Running, Waiting, Terminated) monitor karna.
2. **CPU Allocation:** Ready queue se process chun kar CPU core assign karna (Context Switch).
3. **CPU Deallocation:** Task finish hone par ya time quantum expire hone par CPU reclaim karna.

10.4 Numerical / Interview Thought Process: Context Switch Latency Trace

Context Switch Execution Steps (Process P1 → P2):

1. Timer interrupt trigger hota hai ($t = 0 \mu s$), CPU enters Kernel Mode.
2. Current process P1 ke registers aur program counter PCB1 mein save hote hain ($t = 1 \mu s$).
3. Scheduler agla process P2 choose karta hai ($t = 2 \mu s$).
4. PCB2 se saved state CPU registers mein reload hoti hai ($t = 3 \mu s$).
5. Mode bit 1 set karke P2 execute hota hai ($t = 4 \mu s$). Total overhead = $4 \mu s$ non-productive time!

11. Device Management & File System Management

Computer Science Fundamentals | Module 3: Storage & I/O Subsystem | CS_Fundamental-00

Interview Memory Hook: Hazaaron alag-alag companies ke hardware ke liye developer ko alag code na likhna pade, isliye OS **Device Drivers** deta hai. Aur storage ke raw blocks ko sundar folders mein **File System** badalta hai!

11.1 Device Management (I/O Subsystem)

- **Device Drivers:** Hardware-specific translators jo generic commands (read/write) ko hardware signals mein badalte hain.
- **Uniform Interface:** Application code ko hardware details se decouple karta hai.
- **Activities:** Tracking connected devices, designating I/O controllers, scheduling device access, safe allocation & deallocation.

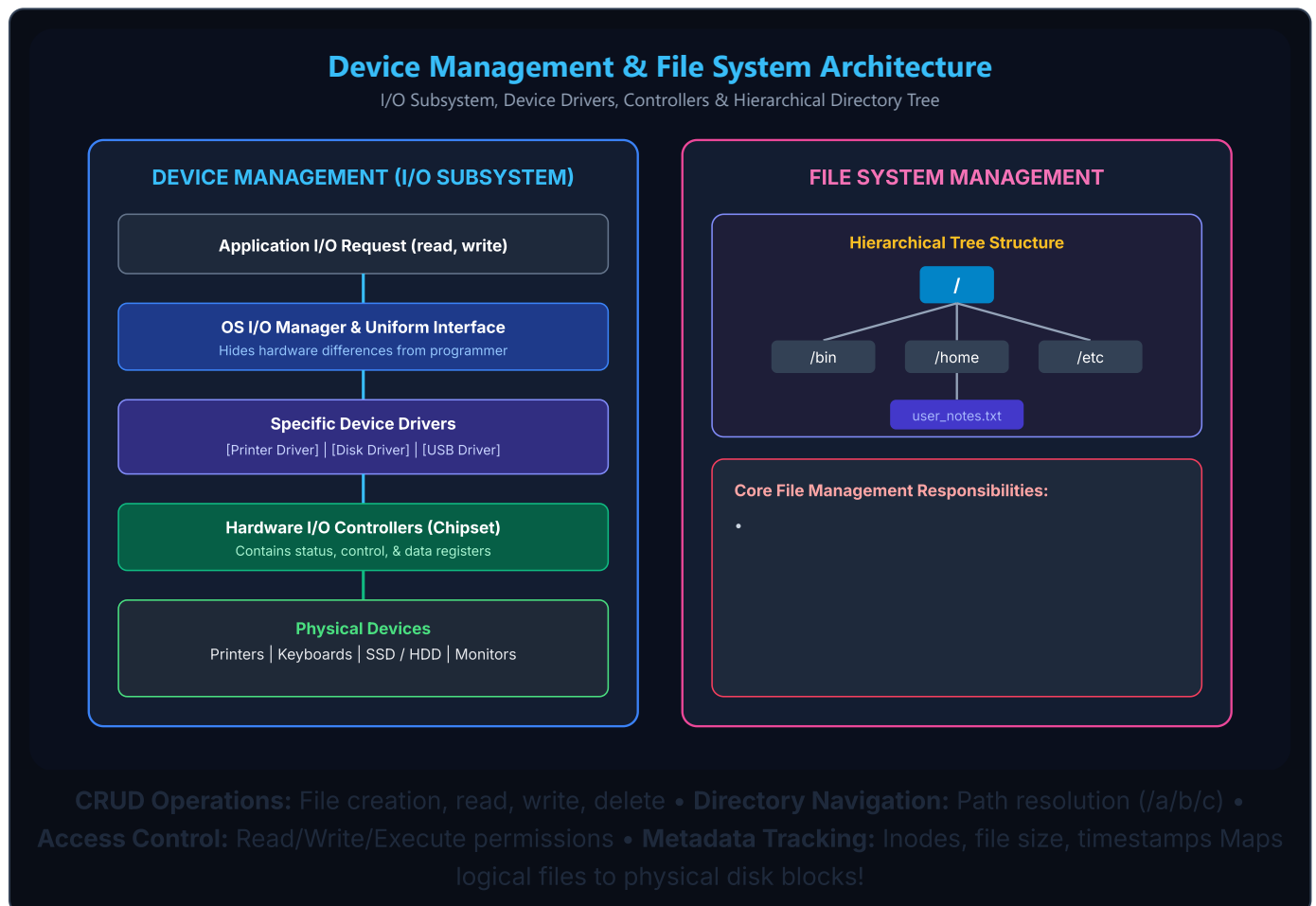


Fig 11.1: Device Driver Subsystem & Hierarchical File System Tree

11.2 File System Management

- **Logical Abstraction:** Physical disk blocks ko logical Files & Directories mein map karna.

- **Hierarchical Tree:** Root directory /, subdirectories, path resolution.
- **File Operations:** Create, Read, Write/Update, Delete, Open, Close.
- **Metadata & Inodes:** File size, permissions (rwx), block pointers, timestamps.

11.3 Step-by-Step Thought Process: File Read Journey (open("notes.txt", "r"))

Execution Trace:

1. **System Call:** User mode app invokes `sys_open()`.
2. **Path Resolution:** Directory traversal: `/` → `home` → `shivam` → `notes.txt`.
3. **Permission & Inode Check:** OS verifies read permission bit.
4. **Block Translation:** Inode maps logical offset to physical disk blocks.
5. **Device Driver I/O:** Disk controller reads blocks into RAM buffer.
6. **File Descriptor Return:** OS returns fd index to user application.

Module 12: User Interfaces, Booting Modes & Specialized OS Functions

Reference Note: Yeh module humare [Module 6: Bootstrap Program & Booting] ([../6_Bootstrap_Program_and_Bootting/README.md](#)) aur [Module 7: Dual-Mode Operation] ([../7_Dual_Mode_Operation/README.md](#)) ke concepts ko aage extend karta hai.

1. User Interface (UI) ya Command Interpreter

User direct bare hardware se communicate nahi kar sakta (jaisa humne [Module 1] ([../1_Introduction_to_OS/README.md](#)) mein dekha tha). OS user aur machine ke beech bridge banta hai through **User Interfaces**:

1.1 Command-Line Interface (CLI)

- **Concept:** Text-based interface jahan user keyboard se text commands enter karta hai (e.g., Linux Shell/Bash, MS-DOS `cmd`, PowerShell).
- **Working:** Shell command read karta hai, parse karta hai, aur corresponding binary program ya System Call execute karta hai (e.g., `mkdir test` $\rightarrow$ `sys_mkdir`).
- **Advantages:**
 - Extremely lightweight (minimal RAM aur CPU usage).
 - Shell scripting ke through repetitive administrative tasks automate ho jate hain.
 - Remote servers (SSH) ke liye standard industry choice.

1.2 Graphical User Interface (GUI)

- **Concept:** WIMP model par based (*Windows, Icons, Menus, Pointer*). User mouse ya trackpad se visual icons par click karta hai (e.g., Windows 11, macOS Aqua, Ubuntu GNOME).
- **Internal Stack:** Desktop Manager $\rightarrow$ Window Server $\rightarrow$ GPU Display Driver $\rightarrow$ OS Kernel.
- **Trade-off:** High memory footprint aur GPU overhead, lekin non-technical users ke liye intuitive aur easy to use.

1.3 Touch-based & Gestural Interfaces

- **Concept:** Mobile aur tablet OS (Android, iOS) jahan physical buttons ki jagah capacitive touch screens aur gestures (swipe, pinch-to-zoom, long-press) use hote hain.
- **Handling:** OS touch controller se continuous interrupt signals generate karta hai aur virtual keyboard handle karta hai.

2. Booting Modes: Cold Booting vs Warm Booting

Concept Reference: [Module 6](../6_Bootstrap_Program_and_Bootting/README.md) mein humne dekha tha ki bootstrap loader ROM se POST run karke OS ko load karta hai.

2.1 Cold Booting (Hard Boot)

- **Definition:** Jab computer **completely OFF state** (Power zero) se start hota hai via physical Power Button.
- **Step-by-Step Flow:**
 1. Power supply turn on hoti hai aur motherboard par Power Good signal generate hota hai.
 2. CPU internal registers reset hote hain aur Program Counter (PC) ROM BIOS/UEFI ke first address par point karta hai.
 3. **POST (Power-On Self-Test):** Motherboard RAM, CPU caches, storage disks, aur keyboard controllers ko physically probe karta hai.
 4. MBR / GPT boot sector se Bootloader RAM mein execute hota hai.
 5. OS Kernel load hota hai aur drivers initialize hote hain.
- **Characteristics:** Slower startup, comprehensive hardware diagnostics.

2.2 Warm Booting (Soft Boot / Reboot)

- **Definition:** Jab machine already powered-on ho aur running state se bina power disconnect kiye system restart kiya jaye (e.g., OS Restart menu, `Ctrl + Alt + Del` , `sudo reboot`).
- **Key Difference:**
- **POST Bypass:** Motherboard power continuously maintain rehti hai, isliye time-consuming hardware diagnostic (POST) skip ho jata hai.
- **Cache & Memory Reset:** RAM ko flush karke fresh OS kernel state re-initialize ki jaati hai.
- **Interview Thought Process:**

$$\text{Boot Time}_{\text{Cold}} = T_{\text{Power Stabilization}} + T_{\text{POST Diagnostic}} + T_{\text{Bootstrap}} + T_{\text{Kernel Init}}$$

$$\text{Boot Time}_{\text{Warm}} = T_{\text{Bootstrap}} + T_{\text{Kernel Init}} \implies \text{Boot Time}_{\text{Warm}} \ll \text{Boot Time}_{\text{Cold}}$$

3. Specialized Auxiliary OS Functions

Operating system sirf CPU scheduling aur memory allocation hi nahi karta; production systems mein yeh 6 critical functions provide karta hai:

3.1 Protection & Security

- **Protection (Internal Boundary):** Ek process doosre process ke memory space ko tamper na kare (Base & Limit registers, Dual-mode kernel ring 0 vs ring 3).
- **Security (External Boundary):** Unauthorized external access ko rokna via User Authentication (passwords/biometrics), Access Control Lists (ACL permissions `chmod 755`), aur storage encryption (BitLocker/LUKS).

3.2 System Performance Monitoring & Optimization

- **Metrics Tracked:**
- **Response Time:** Request submission se first response tak ka time ($T_{\text{response}} = T_{\text{first_output}} - T_{\text{arrival}}$).
- **Throughput:** Number of jobs completed per unit time ($\text{Throughput} = \frac{N}{\Delta t}$).
- **CPU Utilization:** Percentage of time CPU active compute state mein rehta hai.
- **Action:** Agar CPU thrashing ya I/O bottleneck detect ho, to OS process priority adjust karta hai.

3.3 Job Accounting & Resource Billing

- **Definition:** Kaunsa user ya container kitne resources consume kar raha hai uska detailed log maintain karna.
- **Tracked Parameters:**
- Total CPU cycles used.
- Peak RAM footprint in Megabytes.
- Disk Read/Write operations and network bandwidth.
- **Use Case:** Multi-tenant Cloud environments (AWS, Azure) jahan pay-per-use billing models chalte hain.

3.4 Error Detection and Handling (Debugging Support)

- **Hardware Level:** Parity errors, power failure interrupts, memory fault detection.
- **Software Level:** Division by zero (`SIGFPE`), segmentation fault (`SIGSEGV`), buffer overflow.
- **OS Action:** Process dump generate karna (Core Dump), system log record karna (`/var/log/syslog` ya Windows Event Viewer), aur fatal crashes ke time BSOD (Blue Screen of Death) display karna taaki hardware damage na ho.

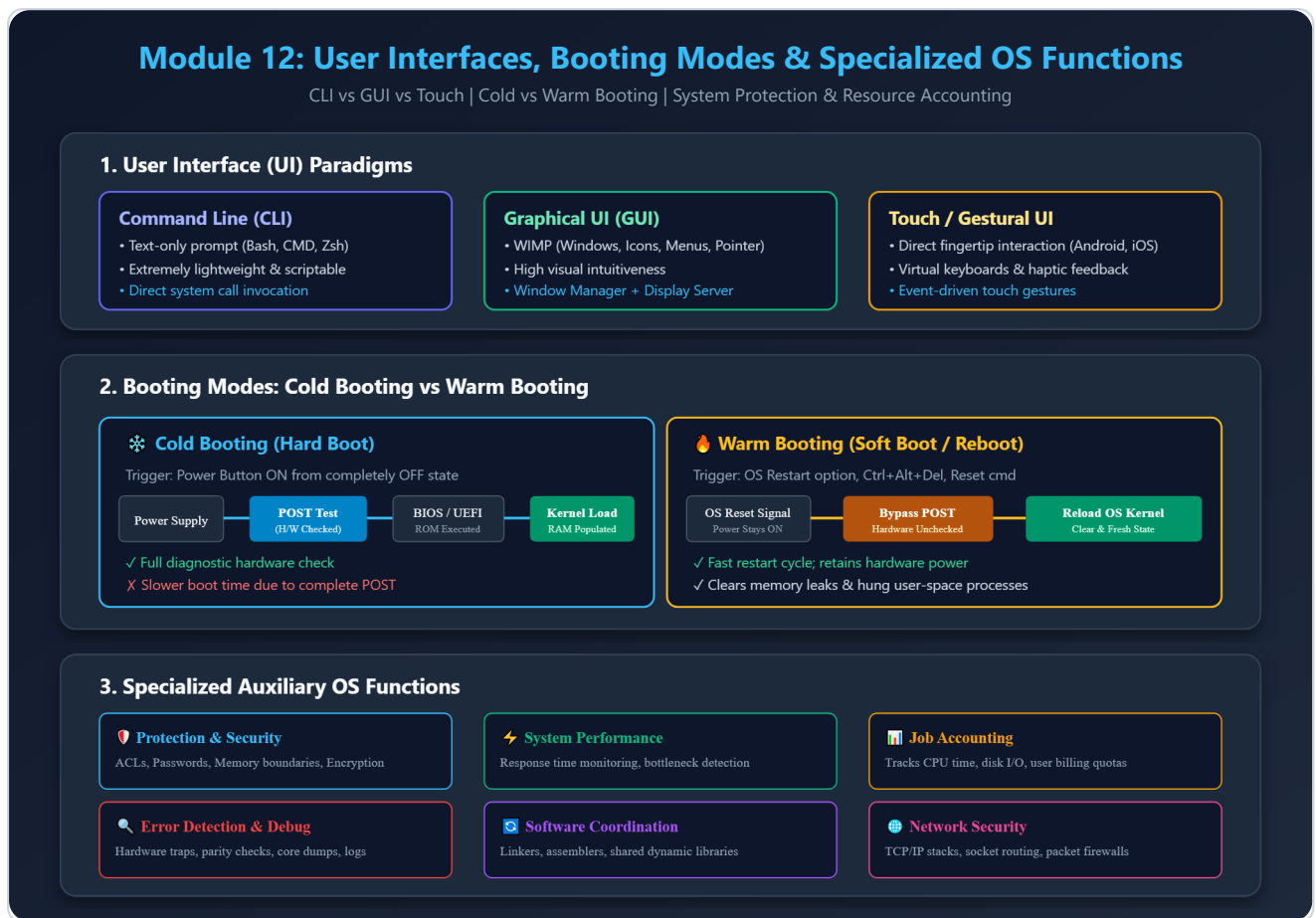
3.5 Software & User Coordination

- **Inter-Software Binding:** OS compilers, linkers, assemblers, aur runtime engines (JVM, Python interpreter) ko shared libraries (`.so`, `.dll`) distribute karta hai.
- Multiple users ke simultaneous program requests ko serialize karta hai bina mutual conflict ke.

3.6 Network Security & Management

- **Networking Stack:** Built-in TCP/IP protocol suite implement karta hai.
- **Socket Interface:** Network packets ko user applications tak route karta hai via network socket APIs.
- **Packet Filtering:** Software firewalls (e.g., `iptables`, Windows Defender Firewall) provide karta hai to inspect and block malicious network traffic.

4. Architectural Summary Diagram



User Interface, Booting & Auxiliary Functions

5. Quick Revision & Interview Q&A

Question	Short High-Yield Answer
Q1: Cold booting aur Warm booting mein main difference kya hai?	Cold boot completely powered-off state se hota hai aur full hardware POST check run karta hai. Warm boot running state se bina power drop kiye hota hai aur POST skip kar deta hai, isliye faster hota hai.
Q2: CLI aur GUI mein memory aur performance ka kya difference hai?	CLI only text buffer use karta hai aur extremely fast/lightweight hota hai. GUI window managers, visual assets, aur GPU rendering pipeline use karta hai, isliye heavier hota hai.
Q3: Job Accounting cloud computing ke liye kyun critical hai?	Multi-tenant clouds (AWS, GCP) mein billing CPU-time, RAM allocation, aur I/O bandwidth consumption par hoti hai; yeh metrics OS job accounting subsystem measure karta hai.

6. PDF Notes

 [\[Download Module 12 Notes \(PDF\)\]\(./12_User_Interface_and_Specialized_OS_Functions.pdf\)](#)

Module 13: Batch Operating Systems & Multiprogramming Systems

Reference Note: Yeh module [Module 10: Processor Management & Scheduling] ([../10_Processor_Management_and_Scheduling/README.md](#)) ke context switching aur CPU utilization concepts ko foundational background deta hai.

1. Evolution & Classification of Operating Systems

Computers ke early days (1950s-1960s) mein programmers direct console par baithe rehte the. Punch cards par code likha jata tha aur ek program execute hone ke baad agla program manually load karna padta tha. Is bottleneck ko solve karne ke liye pehla automated OS architecture aaya: **Batch Operating System**.

2. Batch Operating System

2.1 Core Definition & Structure

- **Definition:** Ek aisa operating system jahan user jobs ko direct CPU par submit nahi karta, balki similar jobs ko group (batch) karke sequentially execute kiya jata hai **without any manual intervention**.
- **Job Structure:**

$$\text{Job} = \text{Program Code} + \text{Input Data} + \text{Control Instructions}$$

- **Role of the Human Operator:**

1. Users apne punch cards ya magnetic tapes operator ko submit karte hain.
2. Operator jobs ko unki requirements ke according categorize karta hai (e.g., Saare FORTRAN jobs ka ek Batch [B_1](#), COBOL jobs ka Batch [B_2](#)).
3. Operator batch system mein feed karta hai.

2.2 Key Advantages

- **Reduced Setup Time:** FORTRAN compiler ek baar load hone par saare FORTRAN programs execute kar deta hai, bar-bar compiler swap karne ki zaroorat nahi padti.
- **Automated Flow:** Ek job finish hone par OS monitor automatically agla job load karta hai.

2.3 Critical Limitations (Drawbacks)

- **No User Interaction:** Execution ke dauran programmer running code se interact nahi kar sakta. Agar runtime bug ya infinite loop aa gaya to user tabhi dekh sakta hai jab poora batch execute ho kar print out aaye (Turnaround time hours ya days mein hota tha).

- **CPU Starvation on I/O:** Agar koi job tape read/write (I/O) kar raha hai, to CPU completely idle baitha rehta tha kyunki memory mein sirf wahi ek job loaded hota tha.

3. Spooling (Simultaneous Peripheral Operation On-Line)

3.1 Spooling ka Concept

- **Full Form:** Simultaneous Peripheral Operation On-Line.
- **Problem Statement:** Slow electromechanical devices (card readers, magnetic tapes, paper printers) CPU ki high speed (10^6 ops/sec) ke muqable bohot slow hote hain (10^2 ops/sec).
- **Working Mechanism:**
 - OS hard disk par ek dedicated buffer/queue maintain karta hai jise **Spool** kehte hain.
 - Jab program `print()` call karta hai, to data directly physical printer par nahi jata, balki high-speed hard disk par spool ho jata hai.
 - Printer independently disk se data FIFO (First In, First Out) order mein fetch karta rehta hai.
- **Difference between Spooling & Buffering:**
 - **Buffering:** Small memory area (RAM) jo single device aur CPU ke beech speed mismatch ko temporarily absorb karta hai.
 - **Spooling:** Disk-based secondary storage jo multiple concurrent processes ke I/O jobs ko queue karta hai aur I/O ko compute ke saath overlap karta hai.

4. Multiprogramming Operating System

4.1 Concept & Working

- **Core Principle:** Main Memory (RAM) mein simultaneously **multiple processes** load karke rakhna taaki CPU kabhi idle na baithe.
- **Execution Flow (Non-preemptive Switch):**
 1. CPU currently Process `P_1` execute kar raha hai.
 2. `P_1` ko disk read ya user input (I/O) ki zaroorat padi, to `P_1` I/O wait state mein chala jata hai.
 3. OS instantly CPU ko interrupt karta hai aur RAM mein already present Ready Process `P_2` ko dispatch kar deta hai.
 4. Jab `P_1` ka I/O complete hota hai, wo wapas Ready Queue mein join kar leta hai.

4.2 Degree of Multiprogramming

- RAM mein simultaneously load hone wale maximum processes ki ginti ko **Degree of Multiprogramming** kehte hain.

4.3 Mathematical Thought Process: CPU Utilization Model

Agar system mein n independent processes loaded hain aur har process apna p fraction of time I/O wait mein spend karta hai:

- Saare n processes ke simultaneously I/O mein hone ki probability:

$$P(\text{All processes in I/O}) = p^n$$

- **CPU Utilization Formula:**

$$\text{CPU Utilization} = 1 - p^n$$

Numerical Example:

Maan lo har process 70% ($p = 0.70$) time I/O wait karta hai:

- **Uniprogramming ($n = 1$):**

$$\text{CPU Util} = 1 - 0.70^1 = 0.30 = \mathbf{30\%}$$

- **Multiprogramming with 4 jobs ($n = 4$):**

$$\text{CPU Util} = 1 - 0.70^4 = 1 - 0.2401 = \mathbf{75.99\%}$$

- **Multiprogramming with 8 jobs ($n = 8$):**

$$\text{CPU Util} = 1 - 0.70^8 = 1 - 0.0576 = \mathbf{94.24\%}$$

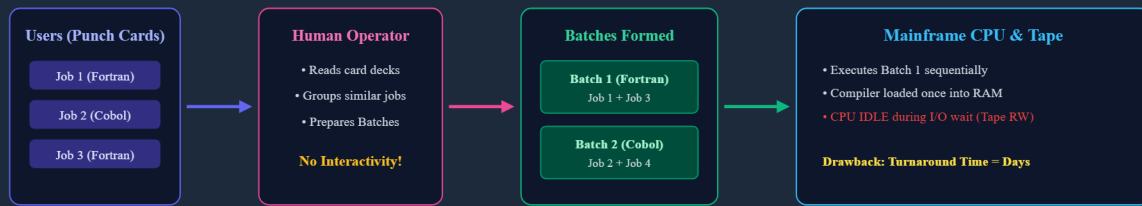
Insight: Multiprogramming se CPU idle time drastically drop ho jata hai aur throughput multi-fold increase ho jati hai.

5. Architectural Diagram

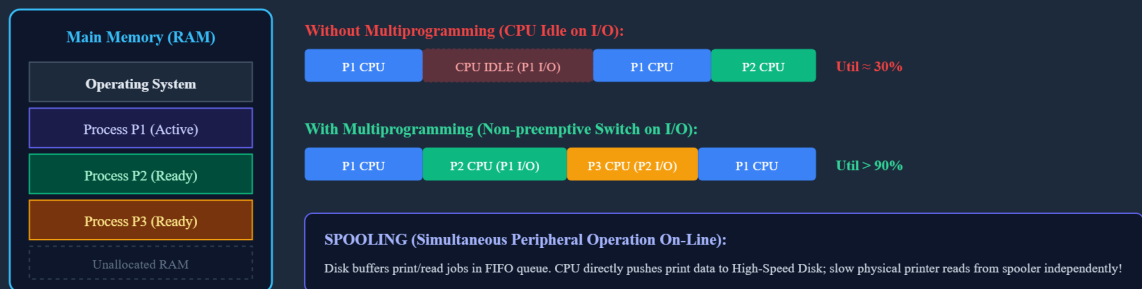
Module 13: Batch Operating Systems & Multiprogramming Architecture

Sequential Job Batches | Spooling Disk Buffering | CPU-I/O Overlap & Multiprogramming

1. Batch Operating System Flow (Operator & Batching)



2. Multiprogramming Architecture & CPU-I/O Concurrency



Batch and Multiprogramming Architecture

6. Comparison: Batch OS vs Multiprogrammed OS

Parameter	Batch Operating System	Multiprogrammed OS
Number of Jobs in RAM	Single job at any moment	Multiple jobs loaded in RAM
CPU Utilization	Extremely low (idle during I/O)	Very high (switches on I/O wait)
Turnaround Time	High (hours to days)	Significantly lower
I/O Handling	Sequential, blocking	Concurrent via Spooling & Overlap
User Interactivity	None	Limited (still primarily non-preemptive)

7. PDF Notes

 [Download Module 13 Notes (PDF)]
(./13_Types_of_Operating_Systems_Batch_and_Multiprogrammed.pdf)

Module 14: Time-Sharing & Real-Time Operating Systems

Reference Note: Yeh module [Module 10: Processor Management & Scheduling] ([../10_Processor_Management_and_Scheduling/README.md](#)) aur [Module 13: Batch & Multiprogrammed OS] ([../13_Types_of_Operating_Systems_Batch_and_Multiprogrammed/README.md](#)) ke context switching principles par build karta hai.

1. Time-Sharing (Multitasking / Fair Share) Operating Systems

1.1 Core Concept

- **Definition:** Time-Sharing ek logical extension hai multiprogramming ka, jahan CPU multiple jobs ke beech itni tezi se switch hota hai (*frequency in milliseconds*) ki har user ko system interactively responsive lagta hai.
- **Fair Share Principle:** CPU kisi bhi single process ko indefinitely monopolize karne nahi deta. Ek hardware timer ke through time ko fixed slices mein divide kiya jata hai.

1.2 Time Quantum (Time Slice q)

- Har ready process ko execution ke liye fixed time allot hota hai jise **Time Quantum (q)** kehte hain.
- **Preemption Rule:**
- Agar Process P_1 time q khatam hone se pehle apna CPU burst complete kar le ya I/O request kare $\rightarrow$ CPU voluntarily release ho jata hai.
- Agar P_1 time q tak execute hota rahe aur complete na ho $\rightarrow$ Hardware timer interrupt trigger hota hai, OS P_1 ko **preempt** karke Ready Queue ke tail par daal deta hai, aur agla process dispatch karta hai.

1.3 Mathematical Thought Process: Time Quantum Sizing Trade-off

Maan lo context switch karne mein δ time lagta hai:

- **Case 1: Quantum q bohot bada ho ($q \rightarrow \infty$):**
- System First-Come-First-Serve (FCFS) jaisa behave karega.
- Interactive users ke liye response time exponentially kharab ho jayega (*Convoy Effect*).
- **Case 2: Quantum q bohot chhota ho ($q \rightarrow 0$):**
- CPU ka majority time sirf context switching overhead mein waste hoga:

$$\text{Overhead Fraction} = \frac{\delta}{q + \delta}$$

- Agar $q = 1 \text{ ms}$ aur $\delta = 0.2 \text{ ms}$, to 16.7% CPU cycles sirf switching mein burn ho jayenge!

- **Golden Rule of Thumb:** Time Quantum q aisa select kiya jata hai ki **80% of CPU bursts** q se chhote hon, aur context switch overhead 1% se kam ho.

2. Real-Time Operating Systems (RTOS)

2.1 Definition & Determinism

- **Definition:** Ek aisa operating system jahan system ki correctness sirf logical output par depend nahi karti, balki **kis time par output deliver hua** uspar bhi depend karti hai.
- **Key Property - Determinism & Bounded Jitter:**

$$\text{Correctness} = (\text{Correct Computation Result}) \wedge (\text{Completion Time} \leq \text{Deadline } D)$$

- General purpose OS (Windows/Linux) **Throughput** maximize karte hain, jabki RTOS **Worst-Case Execution Time (WCET)** aur **Predictability** guarantee karte hain.

3. Hard Real-Time vs Soft Real-Time Systems

3.1 Hard Real-Time System (Zero Tolerance)

- **Deadline Rule:** Ek bhi deadline miss hona system ka **TOTAL FATAL FAILURE** mana jata hai.
- **Architectural Constraints:**
- **No Virtual Memory / Paging:** Secondary disk se page fault aane par variable millisecond delay (jitter) ho sakta hai, isliye paging completely disabled hoti hai.
- Minimalistic deterministic kernel (e.g., VxWorks, QNX Neutrino, RTEMS).
- **Real-World Examples:**
- Cardiac Pacemaker (Heart rate regulation).
- Automotive Airbag Deployment (within 15-30 milliseconds of impact).
- Rocket Propulsion & Missile Guidance Systems.

3.2 Soft Real-Time System (Graceful Degradation)

- **Deadline Rule:** Agar deadline miss ho jaye, to system crash nahi hota, balki output ki **Quality of Service (QoS)** degrade ho jati hai (*utility diminishes*).
- **Architectural Freedom:** Virtual memory, dynamic caching, aur complex multimedia decoders allowed hote hain.
- **Real-World Examples:**
- Live Video Streaming (YouTube, Twitch): Deadline miss hone par frame drop ya buffering aati hai.
- Online Multiplayer Gaming (CS:GO, Valorant): Packet latency increase hone par ping lag aata hai.
- Digital Audio Processing.

4. Architectural Diagram

Module 14: Time-Sharing & Real-Time Operating Systems

Round-Robin Preemption | Time Quantum (q) | Hard vs Soft RTOS Determinism

1. Time-Sharing / Multitasking (Round Robin Preemption)

Circular Ready Queue

```
graph TD; P1((P1)) --> P2((P2)); P2 --> P3((P3)); P3 --> P1
```

Gantt Chart Execution (Time Slice $q = 2\text{ ms}$)

0ms 2ms 4ms 6ms 8ms 10ms 12ms

- Timer interrupt hardware forces CPU context switch har 2ms baad.
- Har user ko illusion milta hai ki computer completely usi ka hai (*Fair Share*).
- ✓ Key Metric: Minimizes Response Time (Interactive systems ke liye best).

2. Real-Time Operating Systems (Hard RTOS vs Soft RTOS)

🚒 Hard Real-Time System (Zero Tolerance)

- Strict Deadlines: Miss deadline = TOTAL CATASTROPHE
- Jitter Tolerance: Exactly 0 microseconds
- No Virtual Memory / Paging (bina page-fault latency ke)

Utility

Deadline

Negative Value (Failure)

Time

Examples: Pacemakers, Airbag Sensor, Missile Avionics

🎮 Soft Real-Time System (Degraded QoS)

- Tolerant Deadlines: Miss deadline = Quality Drop (Lag)
- High priority given to real-time tasks over best-effort
- Virtual memory & dynamic caching supported

Utility

Deadline

Gradual QoS Decay

Time

Examples: Video Streaming (YouTube), VoIP Call, VR Games

Time-Sharing and Real-Time Systems

5. Comprehensive Comparison Matrix (AKTU 10-Marks Target)

Parameter	Multiprogramming OS	Time-Sharing OS	Real-Time OS (RTOS)
Primary Goal	Maximize CPU Utilization	Minimize Response Time	Guarantee Strict Deadlines
Preemption	Usually Non-preemptive (on I/O)	Preemptive (Timer Interrupt q)	Preemptive based on Task Priority
User Interaction	Low to Medium	Very High (Multiple interactive terminals)	Minimal to None (Autonomous sensor events)
Scheduling Algorithm	FCFS, Shortest Job First	Round Robin (RR) with Time Slice	Rate Monotonic (RMS), Earliest Deadline First (EDF)
Failure Metric	Job takes longer	User experiences delay	Catastrophic failure (Hard RTOS)

6. PDF Notes

 [\[Download Module 14 Notes \(PDF\)\]\(/14_Time_Sharing_and_Real_Time_Systems.pdf\)](#)

Module 15: Multiprocessor, Multiuser & Multithreaded Systems

Reference Note: Yeh module [Module 5: Computer System Organization] ([../5_Computer_System_Organization/README.md](#)) aur [Module 10: Processor Management] ([../10_Processor_Management_and_Scheduling/README.md](#)) ke multi-core CPU architecture par build karta hai.

1. Multiprocessor Systems (Parallel / Tightly Coupled)

1.1 Core Definition & Advantages

- **Definition:** Ek aisa computer system jisme **do ya do se zyada physical CPUs / Execution Cores** hote hain jo close communication mein rehte hain aur common system bus, clock, memory, aur peripheral devices share karte hain.
- **Key Advantages:**
 1. **Increased Throughput:** Zyada processors hone se per-unit time mein zyada jobs complete hote hain (Speedup).
 2. **Economy of Scale:** Separate computers kharidne ke bajaye ek hi system mein power supplies, motherboards, aur disks share hoti hain.
 3. **Graceful Degradation (Fault Tolerance / Reliability):** Agar 10 processors mein se 1 processor physically fail ho jaye, to poora system halt nahi hota; system bache hue 9 processors par chalte hue [10\%](#) slower perform karega (*Fail-soft*).

1.2 Flynn's Computer Architecture Taxonomy

Michael J. Flynn ne computer systems ko instruction stream aur data stream ke basis par 4 classes mein divide kiya:

1. **SISD (Single Instruction, Single Data):** Traditional uniprocessor CPU (e.g., standard single-core PC).
2. **SIMD (Single Instruction, Multiple Data):** Single instruction multiple data elements par parallel operate karta hai (e.g., Vector processors, Modern GPUs rendering pixels).
3. **MISD (Multiple Instruction, Single Data):** Theoretical stream (fault-tolerant flight computers).
4. **MIMD (Multiple Instruction, Multiple Data):** Modern multi-core processors aur clusters jahan multiple CPUs independent instructions aur independent data streams handle karte hain.

2. SMP (Symmetric Multiprocessing) vs ASMP (Asymmetric Multiprocessing)

2.1 Symmetric Multiprocessing (SMP)

- **Concept:** Saare processors **peer** (barabar) hote hain. Koi master-slave hierarchy nahi hoti.
- **Execution:** Har processor user program aur kernel code dono execute kar sakta hai.
- **Load Balancing:** Ready Queue se koi bhi available idle CPU next process utha kar execute karne lagta hai.
- **Modern Example:** Modern Linux, Windows Server, macOS (All modern desktop/server OS are SMP).

2.2 Asymmetric Multiprocessing (ASMP)

- **Concept: Master-Slave** architecture follow karta hai.
- **Execution:**
 - Master CPU Operating System kernel code, system calls, aur I/O interrupt handling run karta hai.
 - Slave CPUs sirf user-space programs execute karte hain jo Master unhe assign karta hai.
- **Limitations:** Master processor bottleneck ban jata hai; agar Master fail ho jaye to poora system crash ho jata hai.

3. Mathematical Thought Process: Amdahl's Law (Speedup Limits)

Jab hum multiprocessor system mein cores add karte hain, to kya performance infinitely increase hoti hai?

Gene Amdahl ne iska theoretical formula diya:

$$\text{Speedup } S = \frac{1}{(1 - P) + \frac{P}{N}}$$

- P : Program ka parallelizable fraction ($0 \leq P \leq 1$).
- $(1 - P)$: Strictly serial fraction (jo sirf single core par chal sakta hai).
- N : Number of CPU cores.

Numerical Thought Process:

Maan lo kisi program ka 80% code parallelize ho sakta hai ($P = 0.80$), aur 20% serial hai:

- **With 4 Cores ($N = 4$):**

$$S = \frac{1}{0.20 + \frac{0.80}{4}} = \frac{1}{0.20 + 0.20} = \frac{1}{0.40} = \mathbf{2.5\times}$$

- **With 16 Cores ($N = 16$):**

$$S = \frac{1}{0.20 + \frac{0.80}{16}} = \frac{1}{0.20 + 0.05} = \frac{1}{0.25} = \mathbf{4.0\times}$$

- **With Infinite Cores ($N \rightarrow \infty$):**

$$S_{\max} = \frac{1}{1 - P} = \frac{1}{0.20} = \mathbf{5.0\times}$$

Interview Insight: Serial fraction $(1 - P)$ speedup par hard ceiling laga deta hai. Chahe 1000 cores laga do, 20% serial code hone par speedup 5x se zyada kabhi nahi ho sakta!

4. Multiuser Operating Systems

- **Definition:** Ek aisa OS jo multiple independent human users ko simultaneously system hardware access karne deta hai via separate terminals/SSH sessions (e.g., Linux, UNIX).
- **Critical OS Requirements:**
 1. **User Authentication & Quotas:** Resource consumption limits (disk quota, CPU time cap).
 2. **Memory Protection:** User A ka program User B ke process memory space ko read/write na kar sake.
 3. **File Ownership & Access Matrix:** File permissions (`rwX` for User, Group, Others).

5. Single-Threaded vs Multi-threaded Operating Systems

5.1 Heavyweight Process vs Lightweight Thread

- **Process:** Ek running program jiska apna independent address space (Code, Data, Heap, Stack, Page Table) hota hai. Do processes ke beech context switch bohot costly hota hai (Cache flushing + TLB invalidation).
- **Thread:** Process ke andar ek independent execution stream (Lightweight Process - LWP).
- **Resource Sharing in Threads:**
 - **Shared:** Code Segment, Data Segment, Heap Memory, Open File Descriptors, Network Sockets.
 - **Private per Thread:** Program Counter (PC), CPU Registers, Thread Private Stack.

5.2 Benefits of Multithreading

- **High Responsiveness:** GUI applications mein agar background thread network file download kar raha ho, tab bhi UI thread user clicks ko smoothly handle karta rehta hai.
- **Resource Economy:** Thread creation process creation se `\approx 10\times` faster aur lightweight hoti hai.
- **Multiprocessor Utilization:** Multiple threads different CPU cores par simultaneously parallel execute ho sakte hain.

6. Architectural Diagram

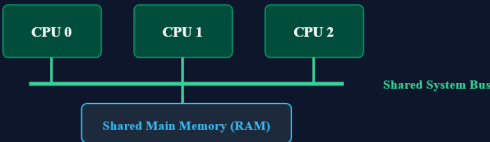
Module 15: Multiprocessor, Multiuser & Multithreaded Systems

SMP vs ASMP | Flynn's Taxonomy | Multiuser Quotas | Heavyweight Process vs Threads

1. Multiprocessor Architecture: Symmetric (SMP) vs Asymmetric (ASMP)

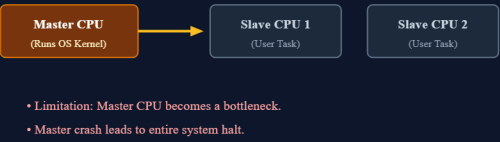
Symmetric Multiprocessing (SMP) - Peer Model

- Saare CPUs equal peer status enjoy karte hain.
- Single shared OS copy; any CPU can run kernel or user tasks.



Asymmetric Multiprocessing (ASMP) - Master-Slave

- Master CPU manages OS scheduling & I/O operations.
- Slave CPUs only execute assigned user code.

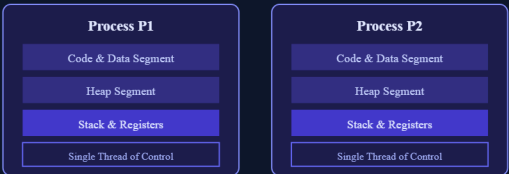


- Limitation: Master CPU becomes a bottleneck.
- Master crash leads to entire system halt.

2. Thread Model vs Heavyweight Process & Multiuser Quotas

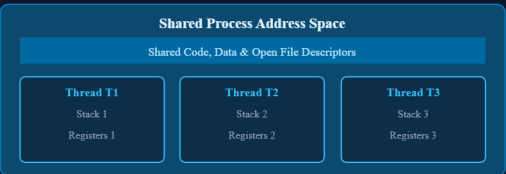
Heavyweight Process Model (Isolated)

- Har process ka separate memory address space hota hai.
- High context switch overhead (Flush TLB + Page tables).



Multithreaded Model (Lightweight Concurrency)

- Threads within same process share Code, Data, Open Files.
- Private state: Har thread ka apna Stack, PC aur Registers hain.



Multiprocessor, Multiuser and Multithreaded Systems

7. PDF Notes

 [Download Module 15 Notes (PDF)](./15_Multiprocessor_and_Distributed_Systems.pdf)

Module 16: Operating System Structures & Architectures

Reference Note: Yeh module [Module 4: Kernel and Operations](../4_Kernel_and_Operations/README.md) ke Monolithic vs Microkernel introduction ko complete architectural depth provide karta hai.

1. Evolution of Operating System Design

Operating system software world ka sabse complex code base hota hai (Linux kernel mein 30+ million lines of code hain). Isliye ise bina structured architecture ke maintain karna impossible hai. History mein 5 prominent architectural approaches evolve hue hain:

1. Simple Structure (MS-DOS)
2. Monolithic Structure (UNIX, Early Linux)
3. Layered Approach (Dijkstra's THE System)
4. Microkernel Architecture (Mach, QNX, Minix)
5. Modular / Loadable Kernel Module (LKM) Structure (Modern Linux, Solaris)

2. Simple Structure (MS-DOS)

2.1 Design & Working

- **Initial Goal:** 1980s mein Intel 8086 processors par limited memory (640 KB) ke liye likha gaya tha.
- **Flaw - No Dual Mode Hardware Isolation:**
- MS-DOS mein User Mode aur Kernel Mode ka hardware protection bit nahi tha.
- User application directly ROM BIOS aur hardware device drivers ko modify kar sakti thi.
- **Drawback:** Agar user application mein koi bug aa jaye ya infinite write execute ho jaye, to poora BIOS aur disk filesystem corrupt ho jata tha.

3. Monolithic Kernel Architecture

3.1 Architectural Working

- **Concept:** Saare core OS subsystems (**Process Scheduler, Virtual Memory Manager, File System, Network Stack, aur Hardware Device Drivers**) ek hi single address space mein privileged **Kernel Mode (Ring 0)** par execute hote hain.
- **Communication:** Subsystems aapas mein direct C function calls ke through communicate karte hain.
- **Advantages:**

- **Blazing Fast Performance:** Zero context-switch ya message-passing IPC latency. Memory mein direct pointer dereference se data transfer hota hai.
- **Disadvantages (Critical AKTU PYQ Topic):**
- **Lack of Fault Isolation:** Kyunki saare drivers kernel mode mein hain, agar third-party sound card ya Wi-Fi driver crash hota hai, to poora operating system crash ho jayega (BSOD / Kernel Panic).
- **Codebase Complexity:** Subsystems tightly coupled hote hain, isliye naya feature add karna ya debug karna risky hota hai.

4. Layered Approach (Dijkstra's Model)

4.1 Concept

- **Strict Hierarchy:** OS ko N layers mein divide kiya jata hai:
- **Layer 0:** Hardware (CPU, Memory, Disks).
- **Layer 1 to $N-1$:** Device drivers, Memory manager, File system.
- **Layer N :** User Interface (Shell / GUI).
- **Fundamental Rule:** Layer i sirf apne niche wali Layer $i-1$ ke functions call kar sakti hai. Kisi bhi layer ko apne upar wali layer ka koi knowledge nahi hota.

4.2 Advantages & Disadvantages

- **Advantages:** Easy debugging and verification. Layer 0 test hone ke baad Layer 1 test hoti hai; agar Layer 1 fail ho to hume pata hai bug Layer 1 mein hi hai.
- **Disadvantages:**
- **Defining Layer Ordering is Difficult:** Memory manager ko disk I/O driver chahiye (backing store ke liye), lekin disk driver ko memory buffers chahiye. Circular dependency resolve karna mushkil hota hai.
- **Performance Overhead:** User request ko 6-7 layers traverse karni padti hai, har layer par function call overhead add hota hai.

5. Microkernel Architecture (Mach, QNX)

5.1 Concept & Philosophy

- **Minimal Privilege Principle:** Kernel space se saare non-essential components bahar nikaal kar **User Space** mein normal background daemons (servers) ki tarah run karaye jate hain.
- **What remains in Microkernel?**
 1. Inter-Process Communication (IPC Message Passing).
 2. Low-level process scheduling.
 3. Minimal virtual memory primitives.

- **What moves to User Space?** File System Server, Network Server, Device Drivers, Process Management Server.

5.2 How Applications Access Services

Jab user program file read karna chahta hai:

1. User App `\rightarrow` sends IPC message to Microkernel.
2. Microkernel `\rightarrow` context switches and routes message to File System Server in User Space.
3. File Server `\rightarrow` checks permissions and sends message to Disk Driver Server.
4. Microkernel `\rightarrow` returns result to User App.

5.3 Pros & Cons

- **Pros (High Reliability & Extensibility):** Agar Wi-Fi driver ya File Server crash ho jaye, to sirf wo user daemon restart hoga; OS kernel unaffected rehta hai.
- **Cons (IPC Overhead):** Multiple user-mode to kernel-mode switches hone ki wajah se monolithic kernel se `10\%-20\%` slower hota hai.

6. Modular / Loadable Kernel Module (LKM) Structure

6.1 Modern Industry Standard (Linux, Solaris)

- **Concept:** Best of both worlds!
- Monolithic kernel jaisi speed: Loaded modules direct kernel space mein run karte hain (zero IPC overhead).
- Microkernel jaisi modularity: Kernel reboot kiye bina naye hardware drivers (`.ko` files) runtime par dynamic link aur unload kiye ja sakte hain via `insmod` aur `rmmmod` .

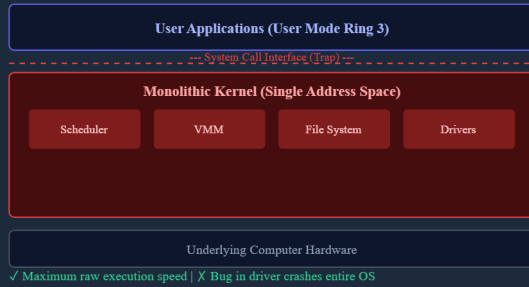
7. Architectural Summary Diagram

Module 16: Operating System Structures & Architectural Approaches

Simple (MS-DOS) | Monolithic Kernel | Layered System | Microkernel | Modular LKMs

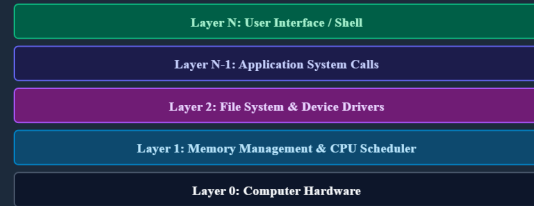
1. Monolithic Kernel Architecture

All services run in privileged Kernel Space (Ring 0)



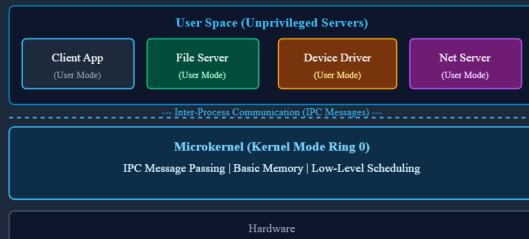
2. Layered Approach (Dijkstra's Model)

Strict hierarchy: Layer N can only call Layer N-1



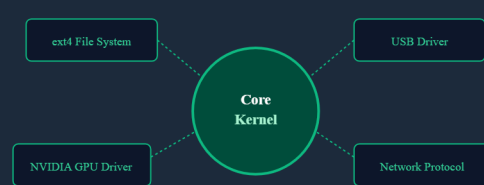
3. Microkernel Architecture (Mach / QNX)

Minimal privileged core; Services run as User Daemons



4. Modular / Loadable Kernel Modules (LKM)

Core kernel loads dynamic modules on-the-fly (Linux/Solaris)



Operating System Structures

8. Comprehensive Comparison Table (AKTU 10 Marks)

Parameter	Monolithic Kernel	Layered System	Microkernel	Modular (LKM)
Driver Location	Kernel Space	Respective Layer	User Space	Kernel Space (Dynamically Loaded)
Communication	Direct Function Call	Strict Layer Calls	IPC Message Passing	Direct Function Call
Reliability	Low (Driver bug halts OS)	Moderate	Very High (Isolated crash)	High (Dynamic unload)
Performance	Fastest	Slower (Stack traversal)	Slowest (IPC overhead)	Fastest
Examples	Original UNIX, MS-DOS	THE System	Mach, QNX, Minix	Linux, macOS, Solaris

9. PDF Notes

 [\[Download Module 16 Notes \(PDF\)\]\(/16_Operating_System_Structures_and_Architectures.pdf\)](#)

Module 17: Advanced Kernel Concepts, Reentrancy & System Calls

Reference Note: Yeh module [Module 7: Dual-Mode Operation](../7_Dual_Mode_Operation/README.md) aur [Module 16: OS Structures](../16_Operating_System_Structures_and_Architectures/README.md) ke foundation par deep-dive execute karta hai.

1. Monolithic Kernel vs Microkernel: The Core Architectural Duel

Modern OS engineering mein yeh sabse famous debate hai (*Tanenbaum-Torvalds Debate*):

Parameter	Monolithic Kernel (Linux)	Microkernel (Mach, QNX, Minix)
Philosophy	"Sab kuch ek sath chalo maximum speed ke liye"	"Sirf minimal primitives kernel mein rakho maximum safety ke liye"
Address Space	Single large kernel address space	Multiple isolated user-mode server spaces
Driver Execution	Privileged Kernel Mode (Ring 0)	Unprivileged User Space (Ring 3)
Communication Mechanism	Direct high-speed function calls	Inter-Process Communication (IPC Message Passing)
Crash Blast Radius	High (Driver crash = Kernel panic)	Minimal (Driver crash = Restart single user process)
Performance Overhead	Minimal (Zero IPC latency)	Moderate (10%-20% IPC context-switch latency)

2. Reentrant Kernel Architecture

2.1 Problem in Non-Reentrant Kernels

- Definition:** Non-reentrant kernel mein agar Process `P_1` system call execute karte hue kernel mode mein hai, to kisi doosre process ya interrupt ko kernel data structures modify karne ki permission nahi hoti thi.
- Race Condition Vulnerability:** Agar CPU context switch kar de jab kernel data structures (e.g., Process Table, Inode Table) half-updated hain, to internal data structure corrupt ho jati hai.
- Old Solution:** Har system call ke entrance par CPU hardware interrupts disable kar deta tha. Iska nuksan yeh tha ki timer interrupts aur critical hardware events drop ho jate the.

2.2 Modern Reentrant Kernel Solution (Linux / Windows)

- **Concept:** Reentrant kernel permits **multiple processes to execute concurrently in kernel mode** without encountering inconsistencies or race conditions.
- **How it Works:**
 1. **Private Kernel Stacks:** Har process ka apna independent kernel-mode stack hota hai jo local variables aur return addresses store karta hai.
 2. **Fine-Grained Locking:** Global data structures par coarse-grained locks lagane ke bajaye fine-grained **Spinlocks** aur **Mutexes** use hote hain.
 3. Example: Process **P_1** process table ke slot 4 ko update kar raha hai, jabki Process **P_2** disk driver ke buffer ko update kar raha hai; dono parallel execute ho sakte hain bina lock conflict ke!

3. Spooling Deep Dive (Simultaneous Peripheral Operation On-Line)

- **Mechanics:** Spooling disk buffer ko use karke multiple processes ke simultaneous requests ko absorb karta hai.
- **Printer Spooling Example:**
 - Jab 5 alag users simultaneously 5 documents print karte hain, to agar direct printing hoti to pages mix ho jate (User A ki 1 line, User B ki 2 line).
 - Spooler har document ko disk par separate spool file ke roop mein store karta hai aur print daemon unhe serial FIFO (First In, First Out) order mein complete print karta hai.

4. Execution Lifecycle of a System Call (Hardware & Software Trace)

Jab user application C code mein `read(fd, buffer, 1024)` execute karti hai, to hardware aur kernel mein yeh 6 stages execute hoti hain:

Stage 1: API Call Wrapper (User Space - Mode Bit = 1)

User program standard C library (`glibc`) ke wrapper function ko invoke karta hai. Library wrapper high-level C arguments ko CPU architecture registers mein convert karta hai.

Stage 2: Register Setup (Syscall Number Mapping)

CPU registers populate kiye jate hain:

- `RAX / EAX` : System Call Number (e.g., `sys_read` has ID `0` in Linux x86_64).
- `RDI` : First argument (File descriptor `fd`).
- `RSI` : Second argument (Buffer pointer).
- `RDX` : Third argument (Byte count `1024`).

Stage 3: Hardware Trap Instruction (Mode Bit Switch 1 \to 0)

CPU special assembly instruction execute karta hai:

- On x86_64: `syscall`
- On legacy 32-bit x86: `int 0x80`
- **Hardware Action:** CPU current Program Counter aur Flags ko kernel stack par save karta hai, Mode Bit ko **1 (User Mode) se 0 (Kernel Mode)** mein flip karta hai, aur CPU execution interrupt descriptor table (IDT) ke predefined kernel address par jump karta hai.

Stage 4: Kernel Dispatcher & `sys_call_table` Lookup

Kernel entry point function (`system_call`) `RAX` register ke index se `sys_call_table` array mein jump karta hai:

```
`c
```

```
// Conceptual Kernel Dispatcher
```

```
if (syscall_nr < NR_syscalls) {
    syscall_handler = sys_call_table[syscall_nr];
    result = syscall_handler(arg1, arg2, arg3);
}
```

```
`
```

Kernel user-supplied buffer pointer ko validate karta hai taaki user program kisi privileged kernel memory ko read/overwrite na kar sake (`access_ok()` check).

Stage 5: Execution of Kernel Service

Kernel subsystem (e.g., VFS / ext4 file system) physical disk controller se data read karta hai aur user buffer memory mein copy karta hai (`copy_to_user`).

Stage 6: Return from System Call (Mode Bit Switch 0 \to 1)

Kernel return value `RAX` register mein store karta hai (bytes read ya negative error code jaise `-EFAULT`), registers restore karta hai, aur `sysret` / `iret` instruction execute karta hai. Mode Bit wapas **0 \to 1** switch ho jati hai aur user application normally resume hoti hai.

```
---
```

5. Architectural Diagram

Module 17: Advanced Kernel Concepts, Reentrancy & System Calls

Reentrant Kernel Concurrency | 6-Stage System Call Execution Lifecycle | Spooling FIFO Architecture

1. Reentrant Kernel Architecture (Concurrent Kernel Mode Execution)

Non-Reentrant Kernel (Race Condition Danger)

- P1 kernel mode mein PCB table modify kar raha hai.
 - Hardware interrupt par P2 kernel mode mein enter ho gaya.
 - Result: Shared kernel data structure **CORRUPT** ho jati hai!
- Workaround (Ancient): Disable ALL interrupts upon entering kernel.
- Drawback: Terrible latency; real-time events missed.

Reentrant Kernel (Modern OS Standard: Linux/Windows)

- Multiple processes simultaneously execute inside kernel mode.
 - Har process ka independent Kernel Stack hota hai.
 - Shared data structures are protected via Spinlocks & Mutexes.
- P1 modifying PCB #4 | P2 simultaneously modifying Inode table #12
- ✓ Zero collision; maximum multi-core CPU parallelism!

2. Step-by-Step Lifecycle of a System Call (Hardware & Software Trace)



3 Parameter Passing Mechanisms in System Calls:

- **1. CPU Registers:** Fast & direct. Parameters CPU registers (RDI, RSI, RDX) mein load hote hain (Linux syscall ABI). Limitation: Registers limited hote hain (<6 parameters).
- **2. Memory Block / Table:** Parameters memory table / structure mein pack kiye jate hain, aur table ka starting memory address CPU register (EBX) mein pass kiya jata hai (Used in Linux & Solaris).
- **3. Program Stack:** Parameters user application stack par push kiye jate hain, aur kernel stack se pop karke retrieve karta hai.

Reentrant Kernel and System Call Lifecycle

6. PDF Notes

 [Download Module 17 Notes (PDF)](/17_Advanced_Kernel_and_System_Calls.pdf)

Module 18: Operating System Services & Core System Components

Reference Note: Yeh module [Module 2: Components & Abstract View] ([../2_Components_and_Abstract_View/README.md](#)) aur [Module 11: Device & File Management] ([../11_Device_and_File_Management/README.md](#)) ke individual modules ko ek integrated overall view provide karta hai.

1. Architectural View of Operating System Services

Operating System hardware ke upar ek abstraction layer create karta hai jisse user programs efficiently run kar sakein. Services ko do broad categories mein divide kiya jata hai:

1.1 User-Oriented Services (Programmer Convenience)

1. **Program Execution:** Program ko secondary storage se RAM mein load karna, execution context initialize karna, aur end hone par resources clean up karna.
2. **I/O Operations:** User applications ko hardware complexity (disk sectors, printer registers) se protect karke simple read/write interface dena.
3. **File System Manipulation:** Files aur directories create, read, write, delete, search, aur file permissions set karna.
4. **Communications:** Same computer par running processes ke beech (Shared Memory, Pipes) ya network ke across (Sockets, RPC) data exchange karna.
5. **Error Detection:** Hardware (memory parity error) ya software level (division by zero, missing file) par error detect karke meaningful recovery action lena.

1.2 System-Oriented Services (Resource Efficiency & Integrity)

1. **Resource Allocation:** Multiple concurrent jobs chalte waqt CPU scheduling, main memory partitioning, aur I/O channel bandwidth distribute karna.
2. **Accounting:** Resource usage statistics track karna (billing, cloud multi-tenancy, capacity planning).
3. **Protection & Security:** Resource boundaries enforce karna taaki koi process unauthorized access na kar sake.

2. The 8 Core Operating System Components

Production operating systems 8 foundational subsystems mein divided hote hain:

2.1 Process Management Subsystem

- Process aur threads create/delete karna.
- CPU Scheduling algorithms (Round Robin, Priority, FCFS) implement karna.
- Inter-Process Communication (IPC) aur process synchronization (Mutexes, Semaphores) provide karna.
- Deadlock detection aur recovery.

2.2 Main Memory Management Subsystem

- Primary memory (RAM) ke har byte ka status track karna (allocated vs free).
- Virtual Memory paging aur page replacement policies (LRU, FIFO) implement karna.
- Memory protection enforce karna via Base aur Limit registers.

2.3 File Management Subsystem

- Hierarchical file directory tree maintain karna (`/home/user/document.txt`).
- Logical files ko physical disk blocks (Sectors/Clusters) par map karna.
- Inodes, FAT tables, aur file metadata maintain karna.

2.4 I/O System Management Subsystem

- **Buffer Caching:** Fast memory buffer use karke device speed mismatch minimize karna.
- **Spooling:** Print jobs aur slow peripheral queues manage karna.
- **Uniform Device Driver Interface:** Naye hardware device ke liye specific driver code provide karna bina core kernel ko alter kiye.

2.5 Secondary Storage Management Subsystem

- Hard Disks aur SSDs par Free Space Management (Bitmaps, Linked Free Lists).
- Disk scheduling algorithms (SSTF, SCAN, C-SCAN) run karke disk head seek time minimize karna.

2.6 Networking & Distributed System Subsystem

- Multiple distributed machines ke resources ko unified system ki tarah present karna.
- TCP/IP protocol suite implement karna.

2.7 Protection & Security Subsystem

- Dual-mode (User vs Kernel Mode) enforce karna.
- User authentication (passwords, biometrics) aur access control policies (ACLs).

2.8 Command-Interpreter System (Shell)

- User input commands parse karke appropriate system calls trigger karna.
- Examples: Linux Bash/Zsh, Windows PowerShell, MS-DOS `cmd.exe` .

3. Architectural Diagram

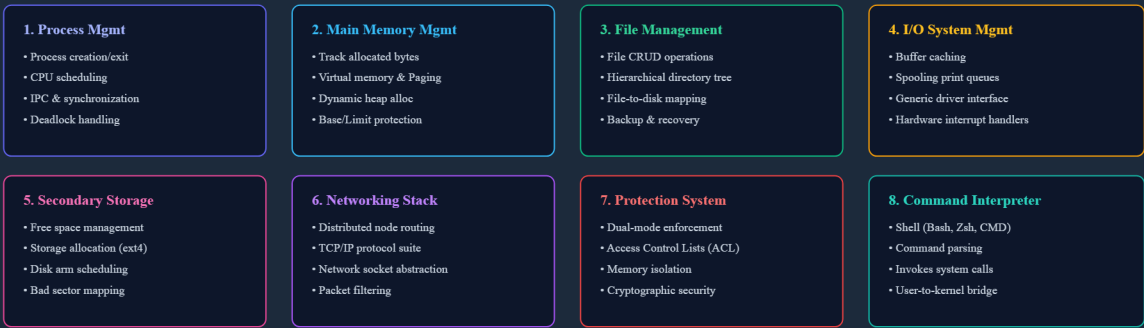
Module 18: Operating System Services & Core System Components

Layered Architecture of OS Services (Slide 130) | 8 Core Architectural Components (Slide 135)

1. Layered View of Operating System Services



2. The 8 Core Operating System Components



Operating System Services and Components

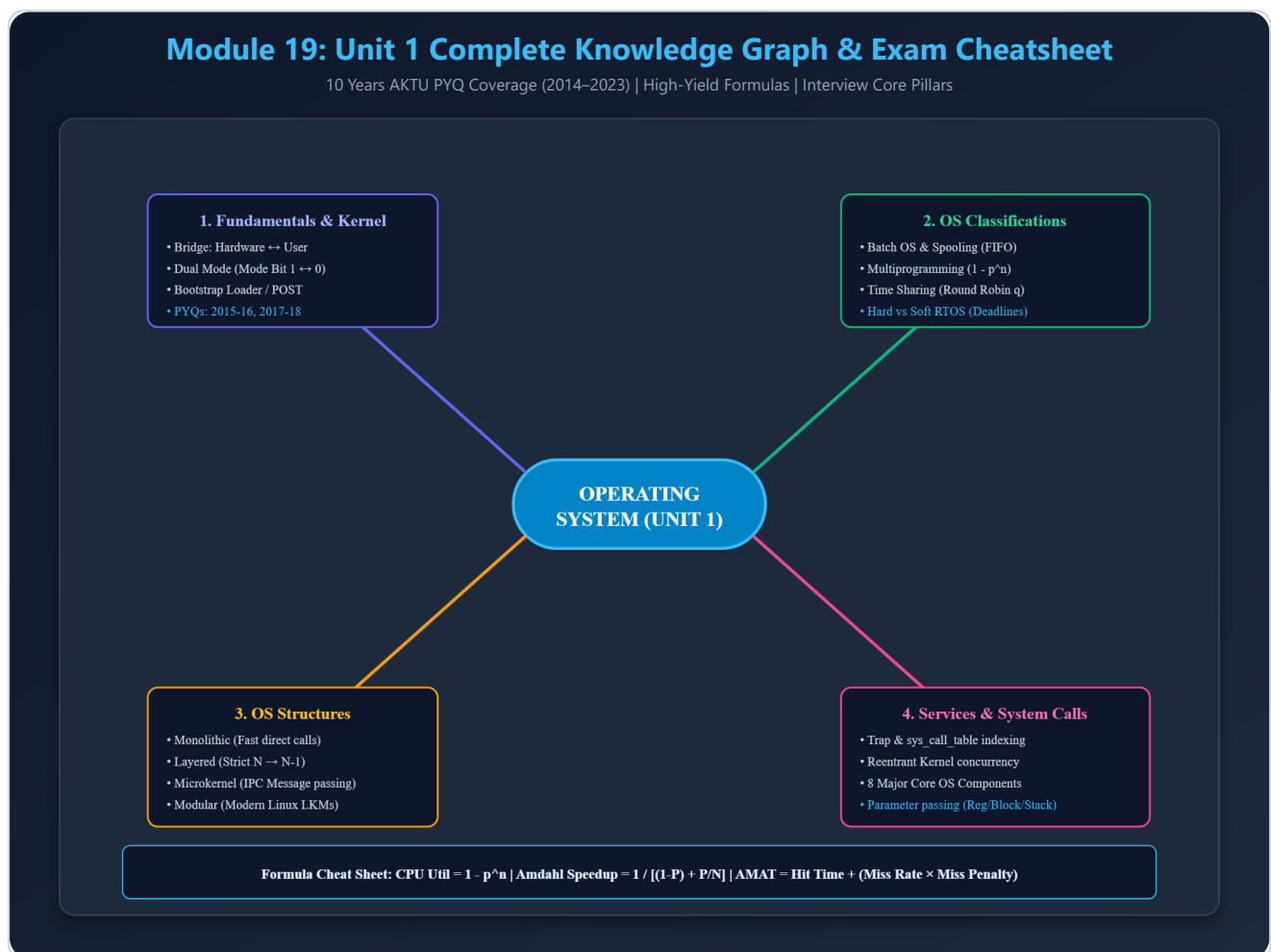
4. PDF Notes

 [Download Module 18 Notes (PDF)](/18_OS_Services_and_System_Components.pdf)

Module 19: AKTU Previous Year Questions (PYQs) & Interview Cheatsheet

Master Exam Document: Yeh module Unit 1 ke sabhi 5 lectures ke **10 Years ke AKTU University Exam Questions (2014–2023)** ke solved model answers, detailed comparisons, aur top tech company interview questions provide karta hai.

1. Unit 1 Complete Mind Map & Knowledge Graph



Unit 1 Knowledge Graph

2. Section A: Solved 2-Marks Short Questions (AKTU Standard)

Q1: What do you mean by Kernel? (AKTU 2015-16, 2018-19)

- **Answer:** Kernel operating system ka **central core component** hai jo computer startup ke baad se system shutdown hone tak permanently Main Memory (RAM) mein resident rehta hai. Yeh hardware aur application software ke beech direct bridge banata hai aur 5 core functions (CPU scheduling, memory management, device control, file operations, security) perform karta hai.

Q2: Define Operating System and list its two main objectives. (AKTU 2016-17, 2017-18, 2021-22)

- **Answer:** Operating System ek system software hai jo computer hardware ko manage karta hai aur user programs ke liye ek execution environment provide karta hai.
- **Two Objectives:**
 1. **User Convenience (User View):** Computer ko use karne mein aasan aur user-friendly banana.
 2. **Resource Efficiency (System View):** Hardware resources (CPU, Memory, I/O) ko optimize aur safely manage karna.

Q3: Define a Multiprogramming System and its principal advantage. (AKTU 2016-17, 2022-23)

- **Answer:** Multiprogramming ek aisa OS architecture hai jahan main memory (RAM) mein ek se zyada active processes simultaneously load rehte hain.
- **Principal Advantage:** Jab currently running process I/O wait mein jata hai, CPU idle rehne ke bajaye doosre process par switch kar jata hai. Isse CPU utilization 30% se badhkar 90%+ ho jati hai.

Q4: What do you mean by Multitasking? (AKTU 2018-19)

- **Answer:** Multitasking (Time-Sharing) multiprogramming ka logical extension hai jahan CPU multiple jobs ke beech fixed time slices (Time Quantum q) par round-robin switch karta hai, jisse multiple users interactively computer share kar sakte hain.

Q5: Briefly define the term Real-Time Operating System (RTOS). (AKTU 2021-22)

- **Answer:** RTOS ek aisa time-bound system hai jahan correctness sirf computational result par hi nahi, balki strict deadline ke andar deliver hone par depend karti hai ($WCET \leq \text{Deadline}$). Examples: VxWorks, QNX.

Q6: What is Spooling? (AKTU 2014-15, 2015-16)

- **Answer:** Spooling (*Simultaneous Peripheral Operation On-Line*) ek mechanism hai jahan slow peripheral devices (printers, card readers) ke I/O data ko high-speed hard disk buffer par FIFO queue mein temporarily store kiya jata hai taaki CPU fast compute kar sake aur I/O overlap ho sake.

Q7: How is a system call handled by the system? (AKTU 2015-16)

- **Answer:** User program library function call karta hai $\rightarrow$ CPU registers mein syscall number load hota hai $\rightarrow$ Software interrupt (Trap / `syscall`) execute hota hai $\rightarrow$ Hardware Mode Bit ko 1 se 0 switch karta hai $\rightarrow$ Kernel `sys_call_table` index lookup karke handler run karta hai $\rightarrow$ Result store karke Mode Bit 0 se 1 flip hota hai aur control user program ko return hota hai.

3. Section B: Solved 5-Marks & 7-Marks Questions

Q8: Differentiate between Interactive and Batch Operating Systems. (AKTU 2017-18, 7 Marks)

Parameter	Batch Operating System	Interactive Operating System
User Interaction	Zero interaction during execution	High continuous interaction via keyboard/mouse
Job Submission	Grouped in batches via punch cards/operator	Direct launch via Terminal (CLI) or GUI
Turnaround Time	Very high (hours to days)	Very low (seconds/milliseconds)
Debugging Ease	Difficult; only post-execution dump available	Immediate runtime feedback and interactive debugging
Primary Goal	Maximize batch throughput	Minimize user response time

Q9: Discuss the difference between Time-Sharing System and Real-Time System. (AKTU 2018-19, 10 Marks)

Parameter	Time-Sharing System	Real-Time System (RTOS)
Primary Metric	Minimize average response time	Strictly meet guaranteed deadlines
Scheduling Model	Round Robin with Time Quantum (q)	Priority-driven (RMS, EDF) preemptive scheduling
Virtual Memory	Heavily used (Paging & Swapping)	Disabled / strictly restricted (to prevent jitter)
Deadline Miss Impact	Mild inconvenience (slight UI lag)	Catastrophic failure in Hard RTOS (fatal crash)
Use Cases	General computing (macOS, Windows, Linux)	Avionics, Automotive ECUs, Medical life support

Q10: Explain the Layered Structure approach of an Operating System with a neat sketch. List its advantages and disadvantages. (AKTU 2014-15, 2017-18, 7 Marks)

Answer & Architecture:

Layered approach mein operating system ko vertically stacked **N** modular layers mein organize kiya jata hai:

- **Layer 0:** Bare Computer Hardware (CPU, Memory, Registers).
- **Layer 1:** Memory Management & Process Dispatcher.
- **Layer 2:** Device Drivers & I/O Channel Controllers.
- **Layer 3:** File System Subsystem.
- **Layer 4 / Layer N:** User Interface (CLI Shell / GUI).

Core Design Rule: Layer *i* sirf apne strictly niche wali Layer *i-1* ke functions call kar sakti hai.

Advantages:

1. **Modularity & Debugging Ease:** System ko layer-by-layer independently verify kiya ja sakta hai. Agar Layer 2 par bug aata hai, to confirm hai ki Layer 0 aur 1 bug-free hain.
2. **Information Hiding:** Har layer internal data structures ko encapsulate karti hai aur upar wali layer ko clean API expose karti hai.

Disadvantages:

1. **Layer Ordering Complexity:** Practical systems mein layers define karna bohot tough hota hai (e.g., virtual memory manager ko disk backing store chahiye, lekin disk driver ko memory buffer chahiye).
2. **Performance Degradation:** Ek simple user request ko 5 layers cross karke traverse karna padta hai, jisse call stack overhead increase ho jata hai.

Q11: Explain in detail about Monolithic and Microkernel Systems. (AKTU 2021-22, 10 Marks)

Detailed Comparison Matrix:

`

[Monolithic Architecture] [Microkernel Architecture]

+-----+ +-----+

User Applications (Ring 3)		User Apps	File Srv	Net Srv
----------------------------	--	-----------	----------	---------

+-----+ +-----+

=== Syscall (Trap) =====		=== IPC Message Passing =====
[Scheduler] [VMM] [File Sys]		[IPC] [Scheduling] [Low Mem]

| [Device Drivers] [Network] | +-----+

+-----+ | Computer Hardware |

| Computer Hardware | +-----+

+-----+

`

1. Kernel Address Space:

- **Monolithic:** Saare OS services (VMM, scheduler, drivers, network) ek single large address space mein privileged ring 0 par execute hote hain.
- **Microkernel:** Sirf basic IPC, low-level scheduling, aur address spaces kernel mode mein hote hain. File systems, networking stacks, aur device drivers unprivileged user mode mein independent servers ki tarah run karte hain.

2. Inter-Service Communication:

- **Monolithic:** Direct C function call (Zero context switch overhead).
- **Microkernel:** IPC Message Passing (User $\rightarrow$ Microkernel $\rightarrow$ Server context switches).

3. Fault Tolerance & Reliability:

- **Monolithic:** Low. Third-party sound card driver crash hone par poora OS crash ho jata hai (BSOD / Kernel panic).
- **Microkernel:** Very High. Driver crash hone par sirf wo individual user process crash hota hai; microkernel bina reboot ke use restart kar deta hai.

4. Real World Examples:

- **Monolithic:** Linux, Original UNIX, MS-DOS.
- **Microkernel:** Mach (macOS foundation), QNX (Used in cars & space missions), Minix.

4. Top 5 Product Company Interview Questions (Google, Amazon, Microsoft)

#	High-Yield Interview Question	1-Minute Rapid Answer
1	Why does an OS need dual-mode operation?	System integrity protect karne ke liye. Agar dual-mode na ho, to koi bhi buggy user program direct hardware control registers overwrite kar dega ya doosre process ki private memory hijack kar lega.
2	What actually happens during a Context Switch?	CPU current process ke registers, Program Counter, aur stack pointer ko uske PCB mein save karta hai; MMU page table pointer update karta hai (TLB flush hota hai); aur new process ke PCB se state CPU registers mein restore karta hai (@approx 1-5 μ s latency).
3	Why don't modern production OS use pure Microkernels?	Pure microkernels mein extreme IPC message-passing context switch overhead hota hai, jisse system monolithic systems ke muqable 10%-20% slower ho jata hai. Modern OS (Linux, Windows) isliye Hybrid / Modular LKM approach use karte hain.
4	What is Amdahl's Law and what is its implication on multicore systems?	Speedup serial fraction $(1 - P)$ par bound hota hai: $S = \frac{1}{(1-P) + P/N}$. Agar program ka 20% code serial hai, to infinite cores lagane par bhi max speedup 5 $\times$ se upar nahi ja sakta.
5	How does Spooling differ from Buffering?	Buffering RAM mein single device aur CPU ke temporary speed mismatch ko handle karta hai. Spooling secondary disk storage par multiple processes ke

#	High-Yield Interview Question	1-Minute Rapid Answer
		concurrent I/O requests ko FIFO queue karta hai.

5. Comprehensive Formula & Rapid Revision Cheatsheet

1. CPU Utilization with Multiprogramming:

$$\text{CPU Utilization} = 1 - p^n$$

(where p = fraction of time process spends in I/O, n = degree of multiprogramming)

2. Amdahl's Speedup Formula:

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

(where P = parallelizable fraction, N = processor cores)

3. Average Memory Access Time (AMAT):

$$\text{AMAT} = T_{\text{hit}} + (\text{Miss Rate} \times \text{Miss Penalty})$$

4. Context Switching Overhead in Time Sharing:

$$\text{Overhead} = \frac{\delta}{q + \delta}$$

(where δ = context switch latency, q = time slice)

6. PDF Notes

 [\[Download Module 19 Notes \(PDF\)\]\(/19_AKTU_PYQs_and_Interview_Cheatsheet.pdf\)](#)